Anne Baanen
Alexander Bentkamp
Jasmin Blanchette
Xavier Généreux
Johannes Hölzl
Jannis Limperg

Lean-zh 项目组 译

逻辑验证漫游指南

***2026* 桌面版**

(2026 年 4 月 11 日)

lean-forward.github.io/
hitchhikers-guide/2026

Copyright © 2018–2026 Anne Baanen, Alexander Bentkamp, Jasmin Blanchette, Xavier Généreux, Johannes Hölzl, and Jannis Limperg. All rights reserved.
本书中文版通过「知识共享：署名-非商业性使用-相同方式共享（CC BY-NC-SA 4.0）」许可协议国际版授权。

目录

前言

形式证明助手是一些旨在帮助用户进行计算机验证证明的软件。我们通常称之为**证明助手**(Proof Assistant)或**交互式定理证明器**(Interactive Theorem Prover)，但一位沮丧的学生创造了「**阻止证明的野兽**(Proof-Preventing Beasts)」一词来形容它，而语音识别软件则偶尔会将「**定理证明器**(Theorem Prover)」误识别为「**恐惧改进器**(Fear Improver)」，敬请留意。

严格证明与形式证明　交互式定理证明有其自己的术语，始于「证明」的概念。**形式证明**(Formal Proof)是用逻辑形式主义表达的逻辑论证。在本文中，「形式」表示「逻辑的」或「基于逻辑的」。逻辑学家，也就是研究逻辑学的数学家，在计算机出现之前的几十年里就在纸上进行形式证明。但如今，形式证明几乎总是使用证明助手来完成。

相比之下，**非形式证明**(Informal Proof)就是数学家通常所说的证明。这类证明通常用 LaTeX 或黑板完成，也被称为「纸笔证明」。其详细程度可能差异很大，诸如「显然」、「清楚地」和「不失一般性」之类的短语将部分证明负担转移给了读者。**严格证明**(Rigorous Proof)则是一种非常详细的非形式化证明。

证明助手的主要优势在于，它们能够运用精确的逻辑，帮助学生构建高度可靠、无歧义的数学陈述证明。它们可以用来证明任意高级的结果，远远超越了玩具示例和逻辑谜题。形式化证明还能帮助学生理解有效定义或有效证明的构成。引用 Scott Aaronson 的话：[1]

> 我还记得自己曾经批改过数百份试卷，学生们一开始只是假设需要证明的内容，或者一页一页地写满胡言乱语，希望在一片混乱中，他们*可能*会偶然说出一些正确的话。

当我们发展一个新理论时，形式化证明可以帮助我们探索它。当我们推广、重构或以其他方式修改现有证明时，形式化证明非常有用，就像编译器帮助我们开发正确的程序一样。形式化证明提供了高度的可信度，使其他人更容易对其进行审查。此外，形式化证明可以构成经过验证的计算工具（例如，经过验证的计算机代数系统）的基础。

[1] https://www.scottaaronson.com/teaching.pdf

成功案例　在数学和计算机科学领域，辅助证明已取得诸多成功。数学形式化领域的一些里程碑式成果包括：Gonthier 等人证明四色定理 [**?**]，Gonthier 等人证明奇数阶定理 [**?**]，Hales 等人证明开普勒猜想 [**?**]，以及 Buzzard 等人定义完美拟态空间 [**?**]。该领域最早的研究始于 20 世纪 60 年代，由 Nicolaas de Bruijn 及其同事在一个名为 AUTOMATH 的系统中开展。[2]

包括 AMD [**?**] 和英特尔 [**?**] 在内的一些公司一直在使用证明助手来验证其设计。在学术界，一些里程碑式的成果包括操作系统内核 seL4 [**?**] 和 CertiKOS [**?**] 的验证，以及经过验证的编译器 CompCert [**?**]、JinjaThreads [**?**] 和 CakeML [**?**] 的开发。

证明助手　全球范围内，有许多正在开发或使用的证明助手。以下列出了一些主要的助手，按其逻辑基础进行分类：

- Set Theory
  集合论：Isabelle/ZF、Metamath、Mizar；
- Simple Type Theory
  简单类型论：HOL4、HOL Light、Isabelle/HOL、PVS；
- Dependent Type Theory
  依值类型论：Agda、Lean、Matita、Rocq（以前叫做 Coq）；
- First-Order Logic
  类 Lisp 的**一阶逻辑**：ACL2。

有关证明助手和交互式定理证明的历史，我们参考了 Harrison、Urban 和 Wiedijk 撰写的资料丰富的章节 [**?**]。

Lean　Lean 是一个证明助手，主要由 Leonardo de Moura（亚马逊网络服务，AWS）自 2012 年以来开发。其数学库 `mathlib` 最初由 Jeremy Avigad（卡内基梅隆大学，CMU）领导开发，现由 Lean 用户社区维护并进一步扩展 [**?**]。[3]

本指南使用 Lean 版本 v4.24.0，`mathlib` 修订版 f897ebcf72cd16f8，以及一些扩展，收集在名为 `LoVelib` 的小型库中。[4] 虽然这只是一个研究项目，还存在一些不完善的地方，但 Lean 非常适合用于交互式定理证明教学，原因如下：

- 它具有强表达能力且非常有趣的，基于**归纳构造演算**（Calculus of Inductive Construction），（一种**依值类型论**（Dependent Type Theory））的逻辑。
- 它扩展了经典公理和**商类型**（Quotient Type），使其可用于验证数学。
- 它包含一个便捷的元编程框架，可用于编写自定义证明的自动化程序。
- 它通过 Visual Studio Code 插件包含一个现代化的用户界面。
- 它具有高度可读性且相当完整的文档。
- 它是开源的。

[2] `https://www.win.tue.nl/automath/`

[3] `https://github.com/leanprover-community/mathlib4`

[4] `https://github.com/lean-forward/logical_verification_2026/raw/main/lean/LoVe/LoVelib.lean`

Lean 的核心库仅包含基本定义和工具。`mathlib` 则提供了更多功能。尽管名为 `mathlib`，但它不仅仅是一个数学库，它在 Lean 的核心库之上提供了大量的自动化功能，满足了人们对现代证明助手的需要。

关于本指南 本指南最初是作为阿姆斯特丹自由大学理学硕士课程《逻辑验证》(LoVe) 的配套课程而设计的。其主要目的是教授交互式定理证明。Lean 是手段，而非目的本身。因此，本指南并非旨在成为全面的 Lean 教程 —— 为此，我们推荐 *Lean 4 定理证明* [**?**]。本指南也不能替代练习和作业。定理证明不是用来看的，它只能通过实践来学习。

具体来说，你的目标是：

- 学习交互式定理证明的基本理论和技术；
- 学习如何使用逻辑作为一种精确的语言来建模系统并描述它们的性质；
- 熟悉一些证明助手可以成功应用的领域，例如函数式编程、命令式编程语言的语义以及数学；
- 培养可应用于大型项目的实践技能（无论是个人项目、硕士、博士项目还是工业界）；
- 能够转换到其他证明助手并应用所学的知识；
- 对该领域有充分的了解，可以开始阅读在国际会议（例如《认证程序与证明》（CPP）和《交互式定理证明》（ITP））或期刊（例如《自动推理杂志》（JAR））上发表的相关科学论文。

具备良好的 Lean 知识后，应该可以轻松迁移到其他基于依值类型论的证明助手，例如 Agda 或 Rocq，或者基于简单类型论的系统，例如 HOL4 或 Isabelle/HOL。

本指南的一个重要特点是，它与 Knuth 的 *TEXbook* [**?**] 一样，并非总是说实话。为了简化阐述，本书提出了一些关于 Lean 的基本但错误的论断。大多数这些论断都会在后续章节中得到纠正。与 Knuth 一样，我们认为「这种故意撒谎的技巧实际上会让你更容易学习这些理念。一旦你理解了一个简单但错误的规则，用它的例外来补充它就不难了。」

本指南附带的 Lean 文件可在公共源码库中找到。[5] 文件的命名方案遵循本指南的章节名，因此，`LoVe07_EffectfulProgramming_Demo.lean` 是与课堂上复习的第 **??** 章「副作用编程」对应的主文件，`LoVe07_EffectfulProgramming_ExerciseSheet.lean` 是习题表，而 `LoVe07_EffectfulProgramming_HomeworkSheet.lean` 是家庭作业表。

我们非常感谢 *《Lean4 定理证明》* [**?**] 和 *《具体语义: Isabelle/HOL 描述》* [**?**] 的作者，他们教会了我们 Lean 和编程语言语义学。他们的许多想法都体现在了本指南中。

[5] https://github.com/Lean-zh/LoVe2026-zh/

我们感谢 Robert Lewis 和 Assia Mahboubi 对本指南的组织和重点提出的宝贵意见。我们感谢 Kiran Gopinathan 和 Ilya Sergey 分享了第 **??** 章脚注 **??** 中提到的轶事，并允许我们进一步分享。我们感谢 Daniel Fabian 设计了本指南的第一个平板电脑优化版本。我们感谢 Paul Chisholm 撰写了相关 Lean 文件中的一些评论。我们感谢 Pietro Monticone 随着 Lean 和 `mathlib` 的发展而帮助更新 Lean 文件。We thank Moritz Roos for completing a proof about the real numbers, which is now included in the main Lean file associated with Chapter **??**. We thank Henrik Böving for his comments on the Lean files associated with the guide. 我们感谢 Moritz Roos 完成了关于实数的证明，该证明现在包含在**??** 一章对应的主 Lean 文件中。我们感谢 Henrik Böving 对与本指南相关的 Lean 文件提出的意见。我们感谢 Mark Summerfield 提出的许多文本建议。最后，我们感谢 Fabian Axer Avila、Chris Bailey、Kevin Buzzard、Paul Chisholm、Dominique Danco、Raufs Dunamalijevs、Wan Fokkink、Lina Gerlach、Alexandra Graß、Robert Lewis、Alexandre Rademaker、Antonius Danny Reyes、Robert Schütz、Kristina Sojakova、Patrick Thomas、Balazs Toth、Huub Vromen、Floris Westerman、Wijnand van Woerkom 和 Yiming Xu 报告了他们在本指南早期版本中发现的错别字和一些更严重的错误。如果您发现本文中存在任何错误，请告知对应的作者。[6]

特殊符号 在本指南中，我们假设你使用 Visual Studio Code 及其「lean4」扩展来编辑`.lean` 文件。Visual Studio Code 允许你通过输入反斜杠 \ 后跟 ASCII 标识符来输入 Unicode 符号。例如，→、∀ 或 ∈ 可以通过输入`\->`、`\fo` 或 `\in` 并按下 Tab 键或空格键来输入。我们将自由使用这些符号。作为参考，我们提供了本指南中使用的主要非 ASCII 符号列表，并为每个符号提供了其对应的 ASCII 表示形式。在 Visual Studio Code 中，按住 Control 键或 Command 键的同时将鼠标悬停在某个符号上，就能看到该符号的不同输入方式。

[6] Jasmin Blanchette (`jasmin.blanchette@ifi.lmu.de`)

¬	\not	∧	\and	∨	\or
→	\->	↔	\<->	∀	\fo
∃	\ex	≤	\<=	≥	\>=
≠	\neq	≈	\~~	×	\x
∘	\circ	∅	\empty	∪	\union
∩	\intersect	∈	\in	↽	\downleftharpoon
○	\bigcirc	←	\<-	↦	\mapsto
⇒	\=>	⟹	\==>	⟦	\[[
⟧	\]]	⋅	\.	α	\a
β	\b	γ	\g	ε	\e
σ	\s	₀	\0	₁	\1
₂	\2	₃	\3	₄	\4
₅	\5	₆	\6	₇	\7
₈	\8	₉	\9		

第一部分

基础

第一章 1

类型与项

我们从学习 Lean 的基础知识开始，首先了解**项**（也称为**表达式**）及其**类型**。

Lean 的逻辑基础是一种丰富的逻辑，叫做**归纳构造演算**，它支持**依值类型**。Lean 的逻辑灵感源自 λ-演算，类似于 Haskell、OCaml 和 Standard ML 等类型化函数式编程语言。即使您从未使用过这些语言，也能从现代语言（例如 C++、Java、Python）中找到许多类似的概念。大致来说：

Lean = 函数式编程 + 逻辑

如果你有数学背景，那么你可能已经了解过函数式编程的大部分关键概念了，有时它们有不同的名称。例如，Haskell 程序：

```
fib 0 = 0
fib 1 = 1
fib n = fib (n - 2) + fib (n - 1)
```

与数学定义非常吻合：

$$fib(n) = \begin{cases} 0 & 若\ n = 0 \\ 1 & 若\ n = 1 \\ fib(n-2) + fib(n-1) & 若\ n \geq 2 \end{cases}$$

在本章中，我们将关注 Lean 语言中一个不包含依值类型的部分，称为**简单类型论**（或**高阶逻辑**）。这大致相当于简单类型的 λ-演算 [**?**]，并添加了一个相等运算符（=）。它是一种抽象的、极其简化的编程语言版本，其函数调用机制预示了函数式编程的出现。

1.1 类型

类型可以是基本类型，例如 $\mathbb{Z}$、$\mathbb{Q}$ 和 `Bool`，也可以是**全函数** $\sigma \to \tau$，其中 σ 和 τ 本身就是类型。类型表明表达式可以求值。它们引入了一个在数学中隐式遵循的

规则。原则上来说，没有什么可以阻止数学家陈述 $1 \in 2$，但类型规则会将其标记为可能有误。

从语义上讲，类型可以被视为集合。我们通常会定义类型 $\mathbb{Z}$、$\mathbb{Q}$ 和 Bool，以便它们忠实地刻画数学家的 $\mathbb{Z}$ 和 $\mathbb{Q}$，以及计算机科学家的布尔值，函数箭头 (→) 也是如此。然而，尽管它们有相似之处，Lean 和数学却是截然不同的语言。Lean 的类型可以被**解释**(Interpreted)为集合，但它们本身并非集合。

高阶类型(Higher-Order Type)是左侧包含嵌套的 → 的箭头 → 的类型，例如类型 $(\mathbb{Z} \to \mathbb{Z}) \to \mathbb{Q}$。此类类型的值是接受其他函数作为参数的函数。因此，$(\mathbb{Z} \to \mathbb{Z}) \to \mathbb{Q}$ 是接受 $\mathbb{Z} \to \mathbb{Z}$ 类型函数作为参数并返回 $\mathbb{Q}$ 类型值的一元函数类型。

1.2 项

简单类型论的**项**(Term)，也称为表达式，包括以下几种：

- **常量** (Constant) c;
- **变量** (Variable) x;
- **应用** (Application) t u;
- **匿名函数** (Anonymous Function) fun x ↦ t（也称为 λ-表达式）。

上面的 t 和 u 表示任意项。我们也可以写作 t : σ 来表示 t 具有类型 σ。

每一种项的具体解释如下：

- 常量 c : σ 是一个类型为 σ 的符号，它的含义在当前的全局**语境**(Context)下是固定的。例如，一个算术理论可能包含常量 0 : ℤ、1 : ℤ、abs : ℤ → ℕ、square : ℕ → ℕ 和 prime : ℕ → Bool。常量包括函数（例如 abs）和谓词（例如 prime）。
- 变量 x : σ 要么是**绑定**(Bound)的，要么是**自由**(Free)的。一个绑定的变量会引用一个匿名函数 fun x : σ ↦ t 中的输入 x。反之，一个自由变量是在局部**语境**(Context)下声明的—这一概念将在下面解释。
- 应用 t u，其中 t : σ → τ 和 u : σ，是一个类型为 τ 的项，它表示将函数 t 应用于参数 u 的结果。例如，如果 t 是 abs，u 是 0，则应用是 abs 0。除非它是一个复杂的项，否则不需要在参数周围加上括号—例如 prime (abs 0)。
- 给定一个项 t : τ，一个匿名函数 fun x : σ ↦ t 表示一个类型为 σ → τ 的**全函数**(Total Function)，它将每个类型为 σ 的输入值 x 映射到函数体 t，其中 x 可能出现在 t中。因此，fun x : ℤ ↦ square (abs x) 表示一个函数，它将 0 映射到 square (abs 0)，将 1 映射到 square (abs 1)，依此类推。变量 x 称为被 fun 所绑定。因此，fun 被称为**绑定器**(Binder)。对于数学家来说，一个更熟悉的语法可能是 x ↦ square (abs x)，没有 fun 关键字。

常量和变量在语法上看起来很相似，但它们被视为不同的。常量是全局声明的，而变量则通过 fun 或其他绑定器在局部引入。

应用和匿名函数互为镜像：匿名函数「构造」一个函数；应用则「解构」一个函数。如果我们将两者结合起来会发生什么？如果我们将一个类型为 σ 的参数 u 应用到匿名函数 fun x : σ ↦ t[x]，其中 t[x] 表示某个可能包含 x 的项，那么我们就会得到项 t[u]。（这是一个稍微简化的视角。实际上，如果t[x] 包含一些绑定器，则可能需要重命名绑定变量以避免捕获u 的自由变量。在 Lean 中，这种重命名会自动进行。）以下是一些应用匿名函数的示例：

$$
\begin{array}{rcl}
(\mathtt{fun}\ n : \mathbb{N} \mapsto n)\ 4 & \text{产生} & 4 \\
(\mathtt{fun}\ n : \mathbb{N} \mapsto \mathtt{square}\ (\mathtt{square}\ n))\ 5 & \text{产生} & \mathtt{square}\ (\mathtt{square}\ 5) \\
(\mathtt{fun}\ y : \mathbb{Z} \mapsto 1)\ 0 & \text{产生} & 1 \\
(\mathtt{fun}\ x : \mathbb{Z} \mapsto (\mathtt{fun}\ y : \mathbb{Z} \mapsto x))\ 1 & \text{产生} & \mathtt{fun}\ y : \mathbb{Z} \mapsto 1
\end{array}
$$

虽然我们的函数是一元函数（即它们只接受一个参数），但我们可以通过嵌套 fun 来构建 *n* 元函数，这需要使用一种名为**柯里化**(Currying)的巧妙技巧。例如，fun x : σ ↦ (fun y : τ ↦ x) 表示类型为 σ → (τ → σ) 的函数，它接受两个参数并返回第一个参数。严格来说，σ → (τ → σ) 接受一个参数并返回一个函数，而该函数又接受一个参数。

应用的方式与之类似：若 K := (fun x : ℤ ↦ (fun y : ℤ ↦ x))，则 K 1 = (fun y : ℤ ↦ 1) 且 (K 1) 0 = 1。函数 K 在 K 1 中，如果只应用于一个参数，则被称为**部分应用**(Partially Applied)，因为它可以接受一个额外的参数。

柯里化是一个非常有用的概念，我们可以省略大多数的括号：

$$
\begin{array}{rcl}
\sigma \to \tau \to \upsilon & \text{表示} & \sigma \to (\tau \to \upsilon) \\
t\ u\ v & \text{表示} & (t\ u)\ v \\
\mathtt{fun}\ x : \sigma \mapsto \mathtt{fun}\ y : \tau \mapsto t & \text{表示} & \mathtt{fun}\ x : \sigma \mapsto (\mathtt{fun}\ y : \tau \mapsto t)
\end{array}
$$

我们还将使用以下缩写：

$$
\begin{array}{rcl}
\mathtt{fun}\ (x : \sigma)\ (y : \tau) \mapsto t & \text{表示} & \mathtt{fun}\ x : \sigma \mapsto \mathtt{fun}\ y : \tau \mapsto t \\
\mathtt{fun}\ x\ y : \sigma \mapsto t & \text{表示} & \mathtt{fun}\ (x : \sigma)\ (y : \sigma) \mapsto t
\end{array}
$$

此外，在匿名函数 fun x : σ ↦ t 中，我们通常可以省略类型注解 : σ：

$$
\mathtt{fun}\ x \mapsto t \quad \text{表示} \quad \mathtt{fun}\ x : \sigma \mapsto t
$$

然后，Lean 会尝试根据函数体 t 和匿名函数周围的语境推断其类型。类型推断可以简化符号，并节省一些按键。

在数学中，通常用中缀语法来书写二元运算符（例如，x + y）。

这样的记法在 Lean 中也是可行的，它其实是柯里化函数应用的语法糖（例如，add x y）。

使用 Lean 的一种方法是用 opaque 命令声明我们需要的类型和常量。考虑以下声明：

```
opaque a : ℤ
opaque b : ℤ
opaque f : ℤ → ℤ
opaque g : ℤ → ℤ → ℤ

#check fun x : ℤ ↦ g (f (g a x)) (g x b)
#check fun x ↦ g (f (g a x)) (g x b)
```

前四行声明了四个常量 (a、b、f、g)，它们可用于构成项。最后两行使用 #check 命令对项进行类型检查并显示其类型。使用传统的数学符号，最后一行的项应写作 $x \mapsto g(f(g(a,x)), g(x,b))$。# 前缀标识诊断命令：这些命令可用于调试，但我们通常不会将它们保存在 Lean 文件中。

1.3 类型检查与类型推断

当 Lean 在解析一个项时，它会检查该项的类型是否正确。在此过程中，如果省略了绑定变量的类型，它就会尝试推断这些变量的类型 —— 例如，fun x ↦ square (square x) 中 x 的类型。

对于简单类型论来说，类型检查和类型推断是可判定的问题。诸如重载（多个常量可以重用相同的名称 —— 例如，0 : ℕ 和 0 : ℝ）等高级特性可能会导致不可判定性 [**?**]。Lean 采用务实的、面向计算机科学的方法，并假设如果存在多种可能的类型，则数字 0、1、2、... 属于 ℕ 类型。

Lean 的类型系统可以表示为一个形式系统。一个**形式系统**(Formal System)由**判断**(Judgment)和用于生成判断的**推导规则**(Derivation Rule)组成。类型判断的形式为 C ⊢ t : σ，表示项 t 在局部语境 C 中具有类型 σ。例如，判断 ⊢ abs : ℤ → ℕ 表示常数 abs 在空的局部语境中具有类型 ℤ → ℕ。

局部语境给出了 t 中未受 fun 绑定的变量的类型。它用于跟踪由 t *外部*的绑定器绑定的变量。如果同一个变量 x 被绑定多次，则最后一次出现的变量会覆盖其他变量。例如，判断 x : ℤ, y : ℕ, y : ℤ ⊢ y : ℤ 表示变量 y 在局部语境 x : ℤ, y : ℕ, y : ℤ 中具有类型 ℤ。

对于简单类型论而言，类型判断由四条类型规则产生，每种项对应一条：

$$\frac{}{C \vdash c : \sigma}\ \textsc{Cst} \quad \text{若 } c \text{ 是全局声明的，类型为 } \sigma \text{ 的常量}$$

$$\frac{}{C \vdash x : \sigma}\ \textsc{Var} \quad \text{若 } x : \sigma \text{ 是出现在 } C \text{ 中最右边的 } x$$

$$\frac{C \vdash t : \sigma \to \tau \qquad C \vdash u : \sigma}{C \vdash t\ u : \tau}\ \textsc{App}$$

$$\frac{C, x : \sigma \vdash t : \tau}{C \vdash (\mathtt{fun}\ x : \sigma \mapsto t) : \sigma \to \tau}\ \textsc{Fun}$$

每条规则都有零个或多个前提（水平线上方）、一个结论（水平线下方），以及可能的一个附加条件（水平线右侧）。前提是类型判断，而附加条件则是规则中出现的数学变量的任意数学条件。为了证明前提，我们需要继续向上推导，我们稍后就会看到。至于附加条件，我们可以运用所有数学知识来证明它们为真。

前两条规则分别标记为 Cst 和 Var，没有前提，但它们有附加条件，这些条件必须满足才能应用规则。后两条规则以一或两个判断作为前提，并产生一个新的判断。Fun 是唯一修改局部上下文的规则：当我们进入一个匿名函数的函数体 t 时，我们需要记录绑定变量 x 的存在，以及它的类型，以便在 t 中遇到 x 时做好准备。

我们可以使用此规则系统来证明给定项的类型正确，方法是：通过逆向（即向上）推导并应用规则，构建一个类型判断的**形式推导**（Formal Derivation）。与现实中的树一样，推导树的根节点位于底部。推导的判断出现在根节点处，每个分支都以应用一个无前提的规则结束。规则的应用由水平线和标签表示。以下类型推导确定项 $\mathtt{fun}\ x : \mathbb{Z} \mapsto \mathtt{abs}\ x$ 在任意局部上下文 C 中具有类型 $\mathbb{Z} \to \mathbb{N}$：

$$\dfrac{\dfrac{\dfrac{}{C, x : \mathbb{Z} \vdash \mathtt{abs} : \mathbb{Z} \to \mathbb{N}}\ \textsc{Cst} \qquad \dfrac{}{C, x : \mathbb{Z} \vdash x : \mathbb{Z}}\ \textsc{Var}}{C, x : \mathbb{Z} \vdash \mathtt{abs}\ x : \mathbb{N}}\ \textsc{App}}{C \vdash (\mathtt{fun}\ x : \mathbb{Z} \mapsto \mathtt{abs}\ x) : \mathbb{Z} \to \mathbb{N}}\ \textsc{Fun}$$

从根节点向上阅读证明，注意局部语境是如何贯穿始终的，以及它是如何被 Fun 规则扩展的。该规则将 fun 绑定的变量移动到局部语境，使得 Var 的应用可以在树的上层进行。如果变量 x 已经在 C 中声明，则在进入 fun 表达式后，它将被 $x : \mathbb{Z}$ 覆盖。

总而言之，类型系统由推导规则组成，这些规则可以 (1) 用其数学变量的任意值进行实例化，以及 (2) 连接起来形成推导树。

下面是第二个示例，这次从一个空的局部语境开始：

$$\dfrac{\dfrac{\dfrac{}{x : \mathbb{Z}, y : \mathbb{Z} : \vdash x : \mathbb{Z}}\ \textsc{Var}}{x : \mathbb{Z} \vdash (\mathtt{fun}\ y : \mathbb{Z} \mapsto x) : \mathbb{Z} \to \mathbb{Z}}\ \textsc{Fun}}{\vdash (\mathtt{fun}\ x : \mathbb{Z} \mapsto \mathtt{fun}\ y : \mathbb{Z} \mapsto x) : \mathbb{Z} \to \mathbb{Z} \to \mathbb{Z}}\ \textsc{Fun}$$

到目前为止，两个例子都是类型良好的。如果我们从一个类型错误的项开始，或者在推导树根部的判断中指定了错误的类型或语境，我们就会发现无法完成推导。推导构成了对项类型良好的证明，并且在给定的语境中具有指定的类型。

上述类型系统仅检查项是否类型良好，并不检查类型是否形式良好。例如，给定一元类型构造子 List, List $\mathbb{Z}$（整数列表的类型）是形式良好的，而 $\mathbb{Z}$ List 和

`List List` 是形式不良的。对于简单类型理论，形式良好性检查很容易：只应使用声明的类型构造子，并且每个 *n* 元类型构造子都应传递恰好为 *n* 类型的参数。

1.4 类型居留

Type Inhabitation Inhabitant

给定一个类型 σ，**类型居留**问题是指在空的局部语境中寻找该类型的一个**居留元**，即一个类型为 σ 的项。这练习看似毫无意义，但正如我们将在第 **??** 章中看到的那样，这个问题与寻找命题的证明密切相关。类似「寻找一个类型为 σ 的项」这种看似简单的练习，其实是掌握定理证明的好方法。

虽然这个问题总体上是不可判定的，但只要策略得当，我们就能取得很大进展。要创建给定类型的项，请从占位符 _ 开始，并递归地应用以下两个步骤的组合：

1. 如果类型的形式为 σ → τ，则可能的居留项是一个匿名函数，形式为 `fun x : σ ↦ _`，其中 _ 是一个类型为 τ 的缺失项的占位符。Lean 会将 _ 标记为错误；如果在 Visual Studio Code 中将鼠标悬停在该项上，提示将显示缺失项的类型以及在局部语境中声明的任何变量。
2. 给定一个类型 σ（可以是函数类型），你可以使用任何常量 c 或类型为 $\tau_1 \to \cdots \to \tau_n \to \sigma$ 的变量 x 来构建一个类型为 σ 的项。对于每个参数，你需要放置一个占位符，从而得到 c _ … _ 或 x _ … _。

占位符可以递归地使用相同的策略来消除。

作为示例，我们将应用该策略来查找以下类型的项：

$$(\alpha \to \beta \to \gamma) \to ((\beta \to \alpha) \to \beta) \to \alpha \to \gamma$$

最初，只有步骤 1 适用，其中：

$$\sigma := \alpha \to \beta \to \gamma \quad \text{且} \quad \tau := ((\beta \to \alpha) \to \beta) \to \alpha \to \gamma$$

（回想一下，→ 是右结合的。）这得到了项 `fun f ↦ _`，它拥有正确的类型，但左边有一个占位符。由于参数 f 的类型为 σ，它是一个函数类型，因此使用名称 f 来表示它是合理的。然后，我们继续递归地处理类型为 τ 的占位符。同样，只有步骤 1 可行，因此我们最终得到项 `fun f g ↦ _`，其中 g 的类型为 (β → α) → β，而占位符的类型为 α → γ。步骤 1 的第三次应用得到 `fun f g a ↦ _`，其中 a 的类型为 α，占位符的类型为 γ。

此时，步骤 1 不再适用。让我们看看步骤 2 是否可行。占位符周围的语境包含以下变量：

$$f : \alpha \to \beta \to \gamma,\ g : (\beta \to \alpha) \to \beta,\ a : \alpha$$

回想一下，我们正在尝试构建一个 γ 类型的项。我们唯一能用来实现这一点的变量是 f：它接受两个参数并返回一个 γ 类型的值。因此，我们用项 f _ _ 替换占

位符，其中，两个新的占位符代表两个缺失的参数。将所有内容放在一起，我们现在得到了项 `fun f g a ↦ f _ _`。

根据 f 的类型，占位符的类型分别为 α 和 β。第一个占位符很容易填充，同样使用步骤 2，只需提供 α 类型的 a，不带任何参数即可。对于第二个占位符，我们将步骤 2 与变量 g, 结合使用，它是 β 的唯一来源。由于 g 接受一个参数，因此我们必须提供一个占位符。这意味着我们当前的项是 `fun f g a ↦ f a (g _)`。

我们快完成了。剩下的唯一占位符的类型是 β → α，它是 g 的参数类型。应用步骤 1，我们将占位符替换为 `fun b ↦ _`，其中 _ 的类型为 α。在这里，我们可以简单地提供 a。我们的最终项是 `fun f g a ↦ f a (g (fun b ↦ a))`。

上述推导过程冗长乏味，但却具有确定性：在每个点，步骤 1 或步骤 2 中的其中一项适用，但并非同时适用。一般来说，情况并非总是如此。对于其他一些类型，我们可能会遇到死胡同，需要回溯。我们也可能会完全失败，无处可回溯。

关键在于，项应始终保持语法正确。我们在 Visual Studio Code 中唯一应该看到的红色下划线应该出现在占位符下方。一般来说，开发软件的一个好的原则是，从可以编译的程序开始，进行尽可能小的更改，以获得新的可以编译的程序，并重复此过程，直到程序完成。

1.5 新引入的 Lean 构造总结

在本章和大多数其他章节的末尾，我们都会简要概述本章引入的构造。某些语法具有多重含义，我们将逐步介绍。有关详细信息，请参阅 *Lean 4 参考手册* [**?**]、*Lean 4 定理证明* [**?**] 教程以及 `mathlib` 文档 [1]。

诊断命令

`#check`	检查并打印一个项的类型

声明

`opaque`	声明一个未指定的新常量或类型

[1] https://leanprover-community.github.io/mathlib4_docs/

第二章 2

程序与定理

我们继续学习 Leam 的基础知识，重点关注程序和定理，但暂不进行任何证明。具体来说，我们将回顾如何定义新的类型和函数，以及如何将其预期的性质表述为定理。

2.1 类型定义

Lean 归纳构造演算的一个显著特点是它内置了对归纳类型的支持。Inductive Type **归纳类型**是一种类型，其值是通过应用称为Constructor**构造子**的特殊常量来构建的。归纳类型是在程序中表示非循环数据的一种简洁方式。你可能还知道一些其他的、大致相同的名称，包括Algebraic Data Type**代数数据类型**、Inductive Data Type**归纳数据类型**、Freely Generated Data Type**自由生成的数据类型**、Recursive Data Type**递归数据类型**和Data Type**数据类型**。

2.1.1 自然数的定义

归纳类型的「Hello, World!」示例是自然数类型 `Nat`（ℕ）。在 Lean 中，它可以定义如下：

```
inductive Nat : Type where
  | zero : Nat
  | succ : Nat → Nat
```

第一行向世界声明我们引入了一个名为 `Nat` 的新类型，用于表示自然数。第二行和第三行声明了两个新的构造子，`Nat.zero : Nat` 以及 `Nat.succ : Nat → Nat`，它们可用于构造类型为 `Nat` 的值。按照计算机科学和逻辑学中既定的惯例，计数从 0 开始。第二个构造子让这个归纳定义变得有趣 —— 它需要一个类型为 `Nat` 的参数来产生一个类型为 `Nat` 的值。以下这些项

```
Nat.zero
```

```
Nat.succ Nat.zero
Nat.succ (Nat.succ Nat.zero)
⋮
```

表示类型为 Nat 的不同值 —— 0、0 的后继数、0 的后继数的后继数，以此类推。这种记法被称为**一进制**(Unary)记法，也叫**皮亚诺**(Peano)记法，以逻辑学家**朱塞佩·皮亚诺**(Giuseppe Peano)的名字命名。若想了解在 Lean 中对皮亚诺数的另一种解释（还有一些令人惊叹的电子游戏画面），请参阅凯文·巴扎德（Kevin Buzzard）的文章《计算机能证明定理吗?》。[1]

类型声明的通用格式为

```
inductive 类型名 (参数₁ : 类型₁) ... (参数ₖ : 类型ₖ) : Type
  where
  | 构造子名称₁ : 构造子类型₁
    ⋮
  | 构造子名称ₙ : 构造子类型ₙ
```

对于自然数，我们可以让 Lean 使用大家熟悉的名称 0、1、2 等等，实际上，预定义的 ℕ 类型就提供了这样的语法糖。不过，使用更详细的表示法能让我们更清楚地了解 Lean 的类型定义。

在本章配套的 Lean 文件中，Nat 的定义被放在一个命名空间内，通过 namespace MyNat 和 end MyNat 进行界定，以将其作用域限制在文件的某一部分。在 end MyNat 之后，出现的所有 Nat、Nat.zero 或 Nat.succ 都将指向 Lean 预定义的自然数类型。同样，整个文件被放在 LoVe 命名空间下，以避免与 Lean 现有库中的名称冲突。

我们可以在 Lean 的任何位置使用 #print 命令来查看之前的定义。例如，在 MyNat 命名空间内输入 #print Nat，会显示如下信息：

```
inductive LoVe.MyNat.Nat : Type
number of parameters: 0
constructors:
LoVe.MyNat.Nat.zero : Nat
LoVe.MyNat.Nat.succ : Nat → Nat
```

本书侧重于自然数，这也是本书许多内容偏向计算机科学的体现之一。对于数论学家来说，他们可能更关注整数 ℤ 和有理数 ℚ；分析学家则更希望处理实数 ℝ 和复数 ℂ。然而，自然数在计算机科学中无处不在，并且作为归纳类型有着非常简单的定义。正如我们将在第 **??**章中看到的，自然数还可以用来构建其他类型。

[1] http://chalkdustmagazine.com/features/can-computers-prove-theorems/

2.1.2 算数表达式的定义

下一个例子会用到整数。如果我们要实现一个计算器程序或一种编程语言，那么就需要定义一种类型，用来将算术表达式表示为抽象语法树。下面的例子展示了如何在 Lean 中实现这一点：

```
inductive AExp : Type where
  | num : ℤ → AExp
  | var : String → AExp
  | add : AExp → AExp → AExp
  | sub : AExp → AExp → AExp
  | mul : AExp → AExp → AExp
  | div : AExp → AExp → AExp
```

从数学上讲，该定义等价于用以下生成规则来归纳地定义类型 AExp：

1. 对于每个整数 i，项 AExp.num i 都是一个 AExp 值。（直观上，构造子 AExp.num 就是把一个整数「包装」为一个算术表达式。）
2. 对于每个字符串 x，项 AExp.var x 都是一个 AExp 值。
3. 若 e_1 和 e_2 都是 AExp 值，那么AExp.add e_1 e_2、AExp.sub e_1 e_2、AExp.mul e_1 e_2 和AExp.div e_1 e_2 也都是 AExp 值。

上述定义穷尽了所有情况。AExp 所有可能的取值都只能通过生成规则 1 到 3 构造得到。此外，使用不同生成规则构造出的 AExp 值彼此不同。这两条归纳类型的性质可以用 Joseph Goguen 提出的口号来概括：「无垃圾，无混淆」。

将上面 Lean 中简洁的 AExp 定义与实现相同功能的 Java 程序作对比也许会很有启发意义。该 Java 程序包含一个接口和六个实现该接口的类，分别对应于 AExp 类型及其六个构造子：

```
public interface AExp { }

public class Num implements AExp {
    public int num;

    public Num(int num) { this.num = num; }
}

public class Var implements AExp {
    public String var;

    public Var(String var) { this.var = var; }
}
```

```
public class Add implements AExp {
    public AExp left;
    public AExp right;

    public Add(AExp left, AExp right)
    { this.left = left; this.right = right; }
}

public class Sub implements AExp {
    public AExp left;
    public AExp right;

    public Sub(AExp left, AExp right)
    { this.left = left; this.right = right; }
}

public class Mul implements AExp {
    public AExp left;
    public AExp right;

    public Mul(AExp left, AExp right)
    { this.left = left; this.right = right; }
}

public class Div implements AExp {
    public AExp left;
    public AExp right;

    public Div(AExp left, AExp right)
    { this.left = left; this.right = right; }
}
```

在 Python 中，同样的类声明如下：

```
class AExp:
    pass

class Num(AExp):
```

```
    def __init__(self, num):
        self.num = num

class Var(AExp):
    def __init__(self, var):
        self.var = var

class Add(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right

class Sub(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right

class Mul(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right

class Div(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right
```

2.1.3 列表的定义

我们考虑的下一个类型是有限列表：

```
inductive List (α : Type) where
  | nil  : List α
  | cons : α → List α → List α
```

该类型是**多态的**[Polymorphism]：它由一个类型参数 α 参数化，我们可以用具体类型对其进行实例化。例如，`List ℤ` 表示整数列表的类型，`List (List ℝ)` 表示实数列表的列表类型。类型构造子 `List` 以一个类型作为参数并返回一个类型。类似于 Java 的泛型和 C++ 的模板，多态性是一种提供参数化类型的机制。

以下命令显示了构造子的类型：

```
#check List.nil
#check List.cons
```

Lean 的内置列表提供了语法糖来编写列表：

`[]`	表示	`List.nil`
`x :: xs`	表示	`List.cons x xs`
$[x_1, \ldots, x_n]$	表示	$x_1 :: \ldots :: x_n :: []$

和所有其他二元运算符一样，`::` 运算符的结合优先级低于函数应用。因此，`f x :: List.reverse ys` 会被解析为 `(f x) :: (List.reverse ys)`。避免不必要的括号是一种良好的编程习惯，因为括号会降低可读性。此外，在中缀运算符两侧加上空格，有助于表达正确的优先级。

函数式程序员通常会用 `xs`、`ys`、`zs` 这样的名字来表示列表，虽然在 Lean 中 `l` 也很常见。由于列表包含多个元素，使用复数形式是很自然的。例如，猫的列表可以命名为 `cats`，猫的列表的列表可以命名为 `catss`。当我们将一个非空列表表示为表头和表尾时，通常会写作 `x :: xs` 或 `cat :: cats`。

2.2 函数定义

如果我们只想声明一个函数，那么可以使用 `opaque` 命令（章节 **??**）。但通常来说，我们想要**定义**函数的行为，那么可以使用 `def` 命令。由于 Lean 的根基是函数式变成，函数会定义为可求值的数学表达式，而非修改某些状态的指令时程序。与此相应，Lean 并不使用 `while` 或 `for` 来迭代，而是使用递归作为遍历数据的主要机制。

2.2.1 在自然数上递归

我们来看一个简单的例子，看看递归是如何工作的。斐波那契数的 Lean 定义如下：

```
def fib : ℕ → ℕ
  | 0     => 0
  | 1     => 1
  | n + 2 => fib (n + 1) + fib n
```

左侧的模式对应于函数的参数。在这里，`fib` 的声明中只有一个类型为 `ℕ` 的参数，因此我们只需对一个自然数进行**模式匹配**(Pattern-Match)。当输入具有给定的形式时，相应的模式匹配就会被触发。例如，如果输入是 1，那么模式 1 就会匹配它，并计算对应的右

侧表达式。如果输入是 5，则会触发模式 n + 2，此时 n := 3。然后将 n 取值为 3 计算右侧表达式，得到 fib 4 + fib 3。

递归定义的一般形式为：

```
def 名称(参数₁ : 类型₁)...(参数ₘ : 类型ₘ) : 类型
  | 模式₁ => 结果₁
      ⋮
  | 模式ₙ => 结果ₙ
```

参数 *参数* 1 到 *参数* m 不能用于模式匹配，只有在 *类型* 中声明的剩余参数才能进行模式匹配。如果一行中提供了多个模式，它们之间用逗号分隔。模式中可以包含变量，这些变量在对应的右侧表达式中是可见的，也可以包含构造子。

基本的自然数算术运算，如加法、乘法和幂运算，都可以通过递归来定义。当然，这些运算在 Lean 中已经预定义了（分别为 +、* 和 ^），但自己实现一遍也是很有意义的练习。我们先从加法开始：

```
def add : ℕ → ℕ → ℕ
  | m, Nat.zero   => m
  | m, Nat.succ n => Nat.succ (add m n)
```

我们对两个参数同时进行模式匹配，区分第二个参数为零和非零的情况。每一次对 add 的递归调用，都会从第二个参数中剥离出一个 Nat.succ 构造子。Lean 允许我们用 0 和 n + 1 这样的语法糖来代替 Nat.zero 和 Nat.succ n。

我们可以使用 #eval 或 #reduce 来计算 add 应用于数字的结果：

```
#eval add 2 7
#reduce add 2 7
```

两个命令都会在 Visual Studio Code 中打印出 9。#eval 会使用一个经过优化的解释器，而 #reduce 则会使用 Lean 的推理内核，它的效率更低。

当定一个函数时，最佳实践是每次提供一些测试来确保函数的行为符合预期。你甚至可以在 Lean 文件中保留 #eval 或 #reduce 作为文档。

乘法的定义与加法类似：

```
def mul : ℕ → ℕ → ℕ
  | _, Nat.zero   => Nat.zero
  | m, Nat.succ n => add m (mul m n)
```

下划线（_）表示未使用的变量。我们本可以使用一个名字（例如 m），但 _ 能更好地记录我们的意图。

下面的 #eval 命令会打印 14，符合预期：

```
#eval mul 2 7
```

指数（「*m* 的 *n* 次方」）有多种定义的方式。我们的第一个方案在结构上与乘法的定义完全相同：

```
def power : ℕ → ℕ → ℕ
  | _, Nat.zero   => 1
  | m, Nat.succ n => mul m (power m n)
```

我们可以将第一个参数 m 提取出来，并作为 *参数* 放在函数名旁边，位于引入函数类型的冒号之前（排除该参数 m）：

```
def powerParam (m : ℕ) : ℕ → ℕ
  | Nat.zero   => 1
  | Nat.succ n => mul m (powerParam m n)
```

从外部看，这两个定义没有区别。事实上，我们已经在 List 构造子的类型参数 α 中见过这种语法了（第 **??** 节）。

还有另一种定义方式，即先引入一个通用的迭代器，然后使用正确的参数来调用它：

```
def iter (α : Type) (z : α) (f : α → α) : ℕ → α
  | Nat.zero   => z
  | Nat.succ n => f (iter α z f n)

def powerIter (m n : ℕ) : ℕ :=
  iter ℕ 1 (mul m) n
```

iter 函数接收一个类型 α，一个「零」值 z，一个一元函数 f，以及一个自然数 n，并计算出

$$\underbrace{\mathtt{f}\,(\mathtt{f}\,(\cdots(\mathtt{f}}_{\text{n 次}}\ \mathtt{z})\cdots))$$

它是高阶函数的一个例子：一种将另一个函数（此处为 f）作为参数的函数。在函数式编程中，函数被视为与其他对象一样的普通对象，可以用作参数或函数的返回结果。在这里，我们使用 iter 来计算 m 的 n 次幂：

$$\underbrace{\mathtt{mul}\ \mathtt{m}\,(\mathtt{mul}\ \mathtt{m}\,(\cdots(\mathtt{mul}\ \mathtt{m}}_{\text{n 次}}\ 1)\cdots))$$

最后，请注意 powerIter 的定义不是递归的。对于不使用模式匹配的非递归函数，其语法简单如下：

```
def name (参数₁ : 类型₁) ... (参数ₘ : 类型ₘ) : 类型 :=
  结果
```

2.2.2 算术表达式上的递归

回到算术表达式类型，如果我们想在 Java 中实现一个 eval 函数，我们可能会将其作为 AExp 接口的一部分，并在每个子类中实现它。对于 Add、Sub、Mul 和 Div，我们会递归地在 left 和 right 对象上调用 eval。

在 Lean 中，其语法非常简洁。我们定义一个函数，并使用模式匹配来区分这六种情况：

```
def eval (env : String → ℤ) : AExp → ℤ
  | AExp.num i      => i
  | AExp.var x      => env x
  | AExp.add e₁ e₂ => eval env e₁ + eval env e₂
  | AExp.sub e₁ e₂ => eval env e₁ - eval env e₂
  | AExp.mul e₁ e₂ => eval env e₁ * eval env e₂
  | AExp.div e₁ e₂ => eval env e₁ / eval env e₂
```

请注意，此函数是高阶函数：它接收一个函数 env，该函数表示一个将值分配给变量的「**环境**」(Environment)。在 AExp.var 情况下，环境用于求值变量，并在递归情况（AExp.add、AExp.sub、AExp.mul 和 AExp.div）中传递。

也许你会担心 AExp.div 情况下的除以零问题。让我们看看 #eval 对此有何评价，使用一个将 7 分配给所有变量的环境：

```
#eval eval (fun x ↦ 7) (AExp.div (AExp.var "y") (
    AExp.num 0))
```

输出结果是 0。在 Lean 中，除法被方便地定义为一个全函数，当分母为零时返回零。关于为什么这并不危险的清晰解释，请参阅 Buzzard 的博客。[2]

2.2.3 列表上的递归

列表上的递归函数可以用类似的方式定义：

```
def append (α : Type) : List α → List α → List α
  | List.nil,        ys => ys
  | List.cons x xs, ys => List.cons x (append α xs ys)
```

append 函数接受三个参数：一个类型 α 和两个类型为 List α 的列表。

```
#eval append ℕ [3, 1] [4, 1, 5]
#eval append _ [3, 1] [4, 1, 5]
```

通过传入占位符 _，我们让 Lean 从其他两个参数的类型中推断出类型 ℕ。

为了使类型参数 α 成为隐式参数，我们可以将其放在花括号 { } 中：

[2] https://xenaproject.wordpress.com/2020/07/05/division-by-zero-in-type-theory-a-faq/

```
def appendImplicit {α : Type} : List α → List α → List
    α
  | List.nil,       ys => ys
  | List.cons x xs, ys => List.cons x (appendImplicit xs
    ys)

#eval appendImplicit [3, 1] [4, 1, 5]
```

@ 符号可用于使隐式参数显式化。有时这对于指导 Lean 的解析器是必要的。

```
#eval @appendImplicit ℕ [3, 1] [4, 1, 5]
#eval @appendImplicit _ [3, 1] [4, 1, 5]
```

我们可以在定义中使用语法糖，无论是在 => 左侧的模式中，还是在右侧：

```
def appendPretty {α : Type} : List α → List α → List α
  | [],      ys => ys
  | x :: xs, ys => x :: appendPretty xs ys
```

在 Lean 的标准库中，追加函数是一个名为 ++ 的中缀运算符。我们可以用它来定义一个反转列表的函数：

```
def reverse {α : Type} : List α → List α
  | []      => []
  | x :: xs => reverse xs ++ [x]
```

2.3 定理陈述

Lean 既是证明助手又是编程语言的原因在于，我们可以对定义的类型和常量陈述定理，并证明它们成立。在交互式定理证明中，术语**定理**（Theorem）、**引理**（Lemma）、**推论**（Corollary）、**事实**（Fact）、**性质**（Property）以及**真陈述**（True Statement）的使用或多或少是可以互换的。类似地，**命题**（Proposition）、**逻辑公式**（Logical Formula）和**陈述**（Statement）的意思也都是一样的。

在 Lean 中，命题仅仅是 Prop 类型的项。这与一阶逻辑形成对比，在一阶逻辑中，传统上在语法中区分项和公式。可以证明的命题被称为定理（或引理、推论等）；否则，它就是一个非定理或假陈述。数学家有时将术语「命题」作为定理的同义词（例如，「命题 3.14」），但在形式逻辑中，命题也可以是假的。

以下是关于第 **??** 节中定义的加法、乘法和列表反转操作的真陈述示例：

```
theorem add_comm (m n : ℕ) :
    add m n = add n m :=
  sorry
```

```
theorem add_assoc (l m n : ℕ) :
    add (add l m) n = add l (add m n) :=
  sorry

theorem mul_comm (m n : ℕ) :
    mul m n = mul n m :=
  sorry

theorem mul_assoc (l m n : ℕ) :
    mul (mul l m) n = mul l (mul m n) :=
  sorry

theorem mul_add (l m n : ℕ) :
    mul l (add m n) = add (mul l m) (mul l n) :=
  sorry

theorem reverse_reverse {α : Type} (xs : List α) :
    reverse (reverse xs) = xs :=
  sorry
```

其一般形式为

```
theorem name (参数₁ : 类型₁) ... (参数ₘ : 类型ₘ) :
    陈述 :=
  证明
```

`:=` 符号将定理的陈述与其证明隔开。`theorem` 的语法与没有模式匹配的 `def` 命令非常相似，用 `statement` 代替 `type`，用 `proof` 代替 `result`。在上面的例子中，我们放置了标记 `sorry` 作为实际证明的占位符。这个标记字面上是向未来的读者和 Lean 表示道歉，因为缺乏证明。这也是值得 *担心*的一个理由，直到我们设法消除它。在第 **??** 章和第 **??** 章中，我们将看到如何实现这一点。

带有 `sorry` 证明的 `theorem` 命令的直观语义是：「这个命题应该是可以证明的，但我还没有进行证明 —— 抱歉。」有时，我们想表达一个相关的想法，即：「让我们假设这个命题成立。」Lean 为此提供了 `axiom` 命令，通常与 `opaque` 结合使用。例如：

```
opaque a : ℤ
opaque b : ℤ

axiom a_less_b :
    a < b
```

在 opaque 命令之后，除了它们的类型之外，我们没有关于 a 和 b 的任何信息。公理指定了它们应该具有的性质。该命令的通用格式为：

```
axiom name (参数₁ : 类型₁) ... (参数ₘ : 类型ₘ) :
    陈述
```

公理是危险的，因为它们可能导致逻辑不一致，从而使我们能够推导出 False。例如，如果我们添加第二个公理陈述 a > b，我们就可以很容易地推导出 False。交互式定理证明的历史充满了不一致的公理。这里有一个轶事：在 2020 年的「**认证程序与证明**」(Certified Programs and Proofs) 会议上，一篇提交的论文因为一个有缺陷的公理而被拒绝，其中一位同行评审员从中推导出了 False。[3]因此，我们通常会避免使用公理。

从 Lean 的角度来看，带有 sorry 证明的定理实际上就是一个公理，可能会破坏逻辑的一致性。为了防止误解，最好只在开发证明时将 sorry 作为临时措施，而不是将其作为更明确且诚实的 axiom 的替代品。

2.4 新引入的 Lean 构造总结

诊断命令

#eval	使用优化的解释器执行一个项
#print	打印一个常量的定义
#reduce	使用 Lean 的**推断内核**(Inference Kernel)执行一个项

声明

axiom	陈述一个公理
def	定义一个新的常量
inductive	引入一个类型及其构造子
namespace ... end	在**命名作用域**(Named Scope)内收集声明
theorem	陈述一个定理及其证明

证明命令

sorry	代表缺失的证明或定义

[3]该论文的一位作者报告说，该公理现在已经过修订并作为定理推导出来。整个形式化开发现在已不含公理。修订后的论文已在「**计算机辅助验证 2020**」(Computer Aided Verification 2020)会议上被接受 [**?**]。

第三章 3

逆向证明

Tactic Goal Proposition
策略是在**目标**（即要证明的**命题**）上操作的过程，要么完全证明它，要么产生新的子目标，或者失败。当我们陈述一个定理时，定理的陈述就是初始目标。一旦使用合适的策略消除了所有（子）目标，证明就完成了。此处介绍的大多数策略在Theorem Proving in Lean 4
《***Lean 4* 定理证明**》第 5 章中有更详细的说明 [**?**]。

Tactic Backward Goal
策略是一种**逆向**证明机制。它们从**目标**出发，向后推进到已证明的定理。考虑定理 a、a → b、b → c 以及目标 ⊢ c。与类型判断类似，目标用符号 ⊢ 标识。一个非形式的逆向证明如下：

> 为了证明 c，由 b → c 可知，只需证明 b 即可。
>
> 为了证明 b，由 a → b 可知，只需证明 a 即可。
>
> 为了证明 a，我们直接使用 a。 □

Backward Proof it suffices to
逆向证明的显著特征是使用了「**只需证明**」这一短语。请注意我们是如何从一个Goal**目标**变换到另一个目标（⊢ c、⊢ b、⊢ a）的，直到没有目标需要证明为止。相比之下，Forward Proof**正向证明**会从定理 a 开始，每次处理一个定理，最终推导出所需的定理 c：

> 由 a 与 a → b 可得 b。
>
> 由 b 与 b → c 可得 c，正如期望。 □

Forward Proof Theorem Goal
正向证明只操作**定理**，不操作**目标**。我们将在第 **??** 章深入学习正向证明。

非形式化的证明有时会混合使用这两种风格。只要能通过「为了证明……，只需证明……」之类的措辞清晰地识别出Backward**逆向**步骤，这种混合就是可行的。在目标前面加上 ⊢ 符号有助于提醒这些断言尚未被证明。

Natural Deduction
你可能熟悉的另一种表达证明的格式是**自然演绎**。下面给出了对应于上述证明的自然演绎推导：

$$\dfrac{\vdash b \to c \qquad \dfrac{\vdash a \to b \qquad \vdash a}{\vdash b}\,{\to}E}{\vdash c}\,{\to}E$$

从上往下读时，该推导对应于**正向证明**（Forward Proof）；从下往上读时，它对应于**逆向证明**（Backward Proof）。

3.1 策略模式

在第 **??** 章中，每当需要证明时，我们只是简单地使用了 `sorry` 占位符。对于**策略证明**（Tactical Proof），我们现在将写下 by 来进入**策略模式**（Tactic Mode）。在此模式下，我们可以调用一系列**策略**（Tactic）。

策略在目标上操作，目标由我们想要证明的命题 Q 和**局部语境**（Local Context）C 组成。局部语境由形如 x : σ 的变量声明和形如 h : P 的**假设**（Hypothesis）组成。我们用 C ⊢ Q 表示一个目标，其中 C 是变量和假设的列表，而 Q 是目标的**靶点**（Target）。

为了让事情更具体，请考虑以下 Lean 示例：

```
theorem fst_of_two_props :
    ∀a b : Prop, a → b → a :=
  by
    intro a b
    intro ha hb
    apply ha
```

请注意，蕴含箭头 → 是右结合的；这意味着 a → b → a 与 a → (b → a) 相同。直观地说，该陈述的意思是「a 蕴含了 b 蕴含 a」，或者等价地，「a 和 b 蕴含 a」。

我们将逐行审阅这个证明。

```
  by
```

关键字（Keyword）by 表示我们进入了**策略模式**（Tactic Mode），在该模式下我们可以通过**策略**（Tactic）来指定证明。随后是策略，每行一个，并且所有策略的缩进相同。

如果我们将光标置于 by 关键字上，Visual Studio Code 会报告当前的目标仅仅是原始定理的陈述：

```
⊢ ∀a b : Prop, a → b → a
```

接下来的策略

```
    intro a b
```

告诉 Lean 固定两个**自由变量**(Free Variable)a 和 b，它们对应于同名的两个**绑定变量**(Bound Variable)。该策略模仿了数学家在纸上的工作方式：为了证明一个 ∀ 量化的命题，只需证明该命题对于绑定变量的某些任意但固定的值成立即可。目标变为：

```
a b : Prop ⊢ a → b → a
```

其中命题 a 和 b 在目标的**局部语境**(Local Context)(即 ⊢ 符号左侧）中声明。我们通常为自由变量使用与绑定变量相同的名称，正如我们在这里所做的那样，但这不是强制性的。通常，使用唯一的名称并避免遮蔽现有变量是良好的做法。

除了使用两个变量名调用一次 `intro` 之外，我们也可以调用两次 `intro`，即 `intro a` 后面跟着 `intro b`。通常，每个带有 n 个参数的 `intro` 调用都会提取接下来的 n 个量化的变量。

`intro` 策略不仅限于量化变量。在我们的证明脚本中，

```
intro ha hb
```

告诉 Lean 将（来自 a → b → 的）前提 a 和 b 移至局部语境，并称这些假设为 ha 和 hb。

事实上，为了证明一个蕴含式，只需将其左侧作为假设，并证明其右侧即可。于是目标变为：

```
a b : Prop, ha : a, hb : b ⊢ a
```

习惯上会在假设名称前加前缀 h。

最后，策略

```
apply ha
```

告诉 Lean 将名为 ha 的假设 a 与目标 ⊢ a 进行匹配。由于 a 在语法上等价于 a，匹配成功。这就完成了证明。

非形式化地，我们可以用类似纸笔数学的风格，将证明写成如下形式：

> 令 a 和 b 为命题。
>
> 假设 (ha) a 和 (hb) b 为真。
>
> 为了证明 a，我们需要使用假设 ha. □

（数学家可能会使用数字标签，如 (1) 和 (2)，而非具有描述性的假设名称。）

回到 Lean 证明，我们可以通过将变量和假设声明为定理的参数，来避免调用 `intro`，如下所示：

```
theorem fst_of_two_props_params (a b : Prop) (ha : a) (
    hb : b) :
     a :=
  by apply ha
```

目标是：

```
a b : Prop, ha : a, hb : b ⊢ a
```

定理的所有参数在语境中立即生效。就像前面的例子一样，该目标通过 apply ha 即可证明。

下面是一个连续使用多个 apply 的例子：

```
theorem prop_comp (a b c : Prop) (hab : a → b) (hbc : b
    → c) :
    a → c :=
  by
    intro ha
    apply hbc
    apply hab
    apply ha
```

站在数学家的角度，我们可以将最后的证明表述如下：

假设 (ha) a 为真。

为了证明 c，根据假设 hbc，只需证明 b 即可。

为了证明 b，根据假设 hab，只需证明 a 即可。

为了证明 a，我们使用假设 ha。 □

3.2 基本策略

intro 和 apply 策略是策略证明的基础。其他基本策略包括 exact、assumption、rfl 和 ac_rfl。如果我们有足够的耐心使用这些策略进行推理，而不依赖更强大的**证明自动化**(Proof Automation)，它们可以发挥很大作用。它们也可以用来解决各种逻辑谜题。

下面，大而细的方括号 [] 表示可选的语法。

intro

intro [*名字*$_1$ ... *名字*$_n$]

intro 策略将最前面的 ∀ 量化变量或最前面的**前提**(Assumption) a → 从目标的**靶点**(Target)移动到局部语境中。该策略接受一个可选参数，用于指定语境中变量或前提的名称，从而覆盖默认名称。如果提供了多个名称（即 $n > 1$），则会移动多个变量或前提。

给定一个可证明的目标，intro 总是会产生一个可证明的目标。因此，它被称为是**安全**(Safe)的。

apply

`apply` *定理或假设*

`apply` 策略将目标的Target**靶点**与指定定理或假设的结论进行匹配，并将该定理或假设的前提作为新的目标加入。匹配是在Up to Computation**计算等价**的意义下进行的。

我们必须谨慎调用 `apply`，因为它可能将一个可证明的目标转换为不可证明的子目标。例如，如果目标是 `⊢ True`，而我们 `apply` 了定理 `False → True`，则结论 `True` 会与目标的靶点 `True` 匹配，最后我们得到了不可证明的子目标 `⊢ False`。我们称 `apply` 是Unsafe**不安全**的。

exact

`exact` *定理或假设*

`exact` 策略将目标的靶点与指定的定理或假设进行匹配，从而证明该目标。在这种情况我们也经常可以使用 `apply`，但 `exact` 能更好地传达我们的意图。

assumption

`assumption` 策略从局部语境中寻找一个与目标靶点匹配的假设，并将其应用以证明目标。

rfl

`rfl` 策略证明形如 `⊢ l = r` 的目标，其中左右两边 `l` 和 `r` 在Up to Computation**计算等价**的意义下在语法上是相等的。计算首先是指定义的Expansion**展开**，但也包括将匿名函数应用于参数的Reduction**归约**等等。这些Conversion**转换**有传统的名称。下面列出了主要转换及其示例，其中全局语境包含 `def double (n : ℕ) : ℕ := n + n`：

α-转换	`(fun x ↦ f x) = (fun y ↦ f y)`
β-转换	`(fun x ↦ f x) a = f a`
δ-转换	`double 5 = 5 + 5`
ζ-转换	`(let n : ℕ := 2; n + n) = 4`
η-转换	`(fun x ↦ f x) = f`
ι-转换	`Prod.fst (a, b) = a`

将这些转换作为自左向右的重写规则重复应用称为Reduction**归约**；将转换反向应用一次称为Expansion**展开**。

简而言之，为了证明 ⊢ l = r，rfl 策略会展开 l 和 r 中的定义，并执行 β-归约及其他归约。如果在归约过程中，l 和 r 在某一点变得语法上完全相同，则证明成功。通常，rfl 在数学家会说「根据定义」的地方起作用。

ac_rfl

ac_rfl 与 rfl 类似，但它可以用于在**结合律**(Associativity)(例如 (a + b) + c = a + (b + c)) 和**交换律**(Commutativity)(例如 a + b = b + a) 的意义下进行推理。这适用于已注册为具有结合律和交换律的二元运算，例如算术类型上的 + 和 *，以及集合上的 ∪ 和 ∩。我们将在第 **??** 节中看到一个例子。

sorry

我们在第 **??** 章中遇到的 sorry 证明命令可以作为策略在策略证明中的任何位置使用。它「证明」了当前目标，但实际上并未进行证明。请谨慎使用。

3.3 关于连结词和量词的推理

在学习对自然数、列表或其他数据类型进行推理之前，我们必须首先学习如何对 Lean 的逻辑连结词和量词进行推理。让我们从一个简单的例子开始：**合取**(Conjunction)(∧) 的**交换律**(Commutativity)。

```
theorem And_swap (a b : Prop) :
    a ∧ b → b ∧ a :=
  by
    intro hab
    apply And.intro
    apply And.right
    exact hab
    apply And.left
    exact hab
```

此时，我们建议您将光标置于 Visual Studio Code 中的示例之上，以查看证明状态的序列。通过将光标放在每个命令之上或紧随其后，您可以查看该行命令的效果。对于最后一行，Lean 只会报告「No goals」。

该证明是典型的 intro-apply-exact 组合。它使用了以下定理：

```
And.intro : ?a → ?b → ?a ∧ ?b
 And.left : ?a ∧ ?b → ?a
And.right : ?a ∧ ?b → ?b
```

其中问号（?）表示可以被**实例化**(Instantiate)的变量 —— 例如，通过将目标的**靶点**(Target)与定理的**结论**(Conclusion)进行匹配。这些变量被称为**元变量**(Metavariable)。

上述三个定理是与**合取**(Conjunction)相关的**引入规则**(Introduction Rule)和两个**消去规则**(Elimination Rule)。逻辑符号（例如 ∧）的引入规则是结论以该符号为最外层符号的定理。对偶地，消去规则在其前提中包含该符号。对于每个逻辑符号，引入规则告诉我们**如何证明**(How to Prove)一个以该符号为最外层位置的命题。相比之下，消去规则告诉我们该命题**必然是如何被证明的**(How we must have proved)。

在上述证明中，我们应用 ∧ 的引入规则来证明目标 ⊢ b ∧ a，并应用两个消去规则从假设 a ∧ b 中提取 b 和 a。我们在 ⊢ b ∧ a 中通过使用所谓的引入规则来「消去」∧，这听起来可能很奇怪。术语之所以是「反向」的，是因为我们的证明是**逆向**(Backward)的。

问号也可能出现在目标中。它们表示可以任意**实例化**(Instantiate)的变量。在上面的证明过程中，在 `apply And.right` 策略之后，我们得到了目标：

```
a b : Prop, hab : a ∧ b ⊢ ?left.a ∧ b
```

其中 `?left.a` 是一个**元变量**(Metavariable)。策略 `exact hab` 将（靶点中的）`?left.a` 与（`hab` 中的）`a` 进行匹配。这种通过实例化变量使两个项在语法上等价的过程称为**合一**(Unification)。**匹配**(Matching)是合一的一种特殊情况，即其中一个项不包含变量，如本例所示。

顺便提一下，每当元变量出现在目标中时，还会出现额外的子目标，每个元变量的类型也会作为一个子目标（例如 ⊢ `Prop`）。这些令人困惑的子目标仅仅是一个提醒，说明我们需要用正确类型的项来实例化这些元变量。我们通常可以忽略这些子目标。一旦元变量被实例化（通常是在我们解决另一个子目标时），其相关的子目标也会随之消失。

在 Lean 中，**合一**(Unification)是在**计算等价**(Up to Computation)的意义下进行的。例如，项 `(fun x ↦ ?m) a` 和 `b` 可以通过取`?m := b` 来合一，因为 `(fun x ↦ b) a` 和 `b` 在 β-转换的意义下是语法相等的。

下面是定理 `And_swap` 的另一种证明：

```
theorem And_swap_braces :
    ∀a b : Prop, a ∧ b → b ∧ a :=
  by
    intro a b hab
    apply And.intro
    · exact And.right hab
    · exact And.left hab
```

该定理的陈述方式有所不同，将 a 和 b 作为∀ 量化变量，而不是定理的参数。从逻辑上讲，这是等价的，但在证明中我们必须在 `hab` 之外额外**引入**(Intro)a 和 b。

另一个区别是在子证明前面使用了项目符号（·）。当我们面临两个或多个目标需要证明时，通常良好的风格是在每个子证明前面加上一个点。**策略组合子**(Tactic Combinator)·

专注于第一个子目标；其后的策略必须证明该子目标。在我们的例子中，apply And.intro 策略创建了两个子目标：⊢ b 和 ⊢ a。

第三个区别是，我们现在通过**并置**（Juxtaposition），直接将 And.right 和 And.left 应用于**假设**（Hypothesis）a ∧ b，分别获得 b 和 a，而不是等待定理的前作为新的子目标出现。这是在**逆向证明**（Backward Proof）中迈出的微小**正向**（Forward）步骤。同样的语法既可以用于**卸载**（Discharge）（即证明）一个假设，也可以用于**实例化**（Instantiate）一个 ∀ 量词。这种方法的一个好处是，我们避免了可能令人困惑的?left.a 元变量。

在下一个例子中，我们使用并置来实例化 ∀ 量词：

```
theorem f5_if (h : ∀n : ℕ, f n = n) :
    f 5 = 5 :=
  by exact h 5
```

如果 h 是定理 ∀n, f n = n，那么 h 5 就是定理 f 5 = 5。

析取（Disjunction）（∨）的引入规则和消去规则如下：

```
Or.inl : ∀b : Prop, ?a → ?a ∨ b
Or.inr : ∀b : Prop, ?a → b ∨ ?a
Or.elim : ?a ∨ ?b → (?a → ?c) → (?b → ?c) → ?c
```

Or.inl（「左侧引入」）和 Or.inr（「右侧引入」）中的 ∀ 量词可以通过简单的**并置**（Juxtaposition），将定理名称应用于我们要实例化的值来直接实例化。因此，Or.inl False 对应于定理 ?a → ?a ∨ False。这就是**正向**（Forward）风格。

或者，我们可以在形如 ... ⊢ c ∨ d 的目标上调用 apply Or.inl。这会将定理中的 ?a 设为 c，?b 设为 d。新的子目标是 ... ⊢ c。这就是**逆向**（Backward）风格。

Or.inl 和 Or.inr 都是**不安全**（Unsafe）的：如果您应用了两者中错误的一个，或者在证明中过早地应用了其中任何一个，您可能会得到一个不可证明的子目标。如果您考虑可证明的目标⊢ True ∨ False，就可以很容易地看到这一点：apply Or.inr 会产生不可证明的子目标 ⊢ False。

Or.elim 规则乍一看可能违反直觉。本质上，它指出如果我们有 a ∨ b，那么为了证明任意的 c，只需证明在 a 成立时 c 成立，以及在 b 成立时 c 成立即可。您可以将(?a → ?c) → (?b → ?c) → ?c 看作是一个巧妙的技巧，仅使用蕴含来表达析取的含义。

等价（Equivalence）（↔）的引入规则和消去规则如下：

```
Iff.intro : (?a → ?b) → (?b → ?a) → (?a ↔ ?b)
Iff.mp : (?a ↔ ?b) → ?a → ?b
Iff.mpr : (?a ↔ ?b) → ?b → ?a
```

存在量化（Existential Quantification）（∃）的引入规则和消去规则如下：

```
Exists.intro : ∀w, (?P w → (∃x, ?P x))
Exists.elim : (∃x, ?P x) → (∀a, ?P a → ?c) → ?c
```

∃ 的引入规则可以用于为一个存在量词提供一个**证据**(Witness)进行**实例化**(Instantiate) —— 证据是使量词的主体为真的值。例如：

```
theorem Exists_double_iden :
    ∃n : ℕ, double n = n :=
  by
    apply Exists.intro 0
    rfl
```

同样地，我们以**正向**(Forward)方式实例化一个 ∀ 量词：`Exists.intro 0` 是定理 `?P 0 → (∃x, ?P x)`。该规则是**不安全**(Unsafe)的：为 x 选择错误的证体会导致不可证明的目标。例如，如果目标是 ⊢ ∃n, n > 5，而我们选择 3 作为证体，最后会得到不可证明的子目标 ⊢ 3 > 5。

∃ 的消去规则让人联想到 ∨ 的消去规则。事实上，一种富有成效的思考量化 ∃n, ?P n 的方式是，将其看作是一个可能**无穷**(Infinitary)的析取 ?P 0 ∨ ?P 1 ∨ ⋯。类似地，∀n, ?P n 可以被看作是 ?P 0 ∧ ?P 1 ∧ ⋯。

对于真（True），只有一个引入规则：

```
True.intro : True
```

「真」不包含任何信息。如果它作为**假设**(Hypothesis)出现，它是完全没用的，并且没有消去规则可以从中提取任何信息。下面第 **??** 节中描述的 `clear` 策略可以用来移除此类无用的假设。

对偶地，对于假（False），只有一个消去规则：

```
False.elim : False → ?a
```

无法证明「假」，但如果我们以某种方式从某处（例如从假设中）得到了它，那么我们可以推导出任何东西（?a）。

事实上，**否定**(Negation)(Not) 是根据蕴含和「假」来定义的：¬ a 是 a → False 的缩写。直观上，它们的含义相同。我们可以把「非 a」看作是说「a 会导致荒谬的事情（因此非 a)」。

Lean 的逻辑是**经典**(Classical)的，支持 **排中律**(Law of Excluded Middle) 和 **反证法**(Proof by Contradiction)：

```
Classical.em : ∀a : Prop, a ∨ ¬ a
Classical.byContradiction : (¬ ?a → False) → ?a
```

蕴含（→）和**全称量化**(∀)（Universal Quantification）就像是谚语中那种「不叫的狗」。它们没有引入规则或消去规则。相反，对于这两者，`intro` 策略就是引入原理，而**应用**(如 λ-演算中那样)（Application）就是消去原理。例如，给定定理 `hab : a → b` 和 `ha : a`，应用 `hab ha` 就是一个陈述 `b` 的定理。

为了证明涉及连结词和量词的逻辑谜题，我们提倡一种「盲目」的、「电子游戏」式的推理风格，它主要依赖于`intro` 和 `apply` 等基本策略。以下是一些通常奏效的策略：

- 如果目标的靶点是一个蕴含式 `P → Q`，调用 `intro hP` 将 `P` 移动到你的假设中：`..., hP : P ⊢ Q`。
- 如果目标的靶点是一个全称量化 `∀x : σ, Q`，调用 `intro x` 将 `x` 移动到局部语境中：`..., x : σ ⊢ Q`。
- 寻找一个结论与目标的靶点形状相同（可能包含可以匹配的变量）的定理或假设，并对其调用 `apply`。例如，如果目标的靶点是`Q`，而你有一个形如 `hPQ : P → Q` 的定理或假设，请尝试 `apply hPQ`。
- 一个被否定的目标 `⊢ ¬ P` 在计算上等价于 `⊢ P → False`,因此你可以调用 `intro hP` 来产生子目标 `hP : P ⊢ False`。通过调用 `rw [Not]`（在第 **??** 节中描述）来展开否定的定义通常是一个好策略。
- 有时你可以通过输入 `apply False.elim` 将目标替换为 `False` 来取得进展。作为下一步，你通常会 `apply` 一个形如 `P → False` 或 `¬ P` 的定理或假设。
- 当你面临多个选择时（例如在 `Or.inl` 和 `Or.inr` 之间选择），请记住你所做的选择，并在遇到**死胡同**(Dead End)或感觉没有进展时进行**回溯**(Backtrack)。
- 如果你怀疑自己可能遇到了死胡同，请检查在给定的前提下目标是否确实是可证明的。即使你最初陈述的是一个可证明的定理，当前的目标也可能是不可证明的（例如，如果你应用了不安全的规则）。

3.4 关于相等的推理

相等(=)（Equality）也是一个基本的逻辑常量。它的特征由以下引入规则和消去规则给出：

```
  Eq.refl : ∀a, a = a
  Eq.symm : ?a = ?b → ?b = ?a
Eq.trans : ?a = ?b → ?b = ?c → ?a = ?c
Eq.subst : ?a = ?b → ?P ?a → ?P ?b
```

前三个定理是指定 `=` 为**等价关系**(Equivalence Relation)的引入规则。第四个定理是一个消去规则，它允许我们在任意语境（由元变量 `?P` 表示）中进行等量替换。

下面的例子展示了其中一些规则的应用。我们应用 Eq.trans 和 Eq.symm，利用等式 a = b 和 c = b 来证明 a = c：

```
theorem Eq_trans_symm {α : Type} (a b c : α)
      (hab : a = b) (hcb : c = b) :
    a = c :=
  by
    apply Eq.trans
    · exact hab
    · apply Eq.symm
      exact hcb
```

由于这种重写操作非常常见，Lean 提供了一个 rw 策略来实现相同的结果。如果适用，该策略还会自动应用 rfl：

```
theorem Eq_trans_symm_rw {α : Type} (a b c : α)
      (hab : a = b) (hcb : c = b) :
    a = c :=
  by
    rw [hab]
    rw [hcb]
```

关于解析的说明：相等的结合优先级高于逻辑连结词。因此，a = b ∧ c = d 被解读为 (a = b) ∧ (c = d)。

3.5 重写策略

重写策略 rw 及其相关的 simp 进行等量替换。它们将等式作为自左向右的重写规则，将出现的左侧项替换为右侧项。

默认情况下，它们在目标的靶点上操作，但也可以通过 at 关键字来重写指定的假设：

at $h_1 \ldots h_n$	重写指定的假设
at *	重写所有假设和目标

rw

rw [*定理或常量* $_1$, ..., *定理或常量* $_n$] [at *位置*]

rw 策略使用一个或多个等式作为自左向右的重写规则来重写目标。它会搜索第一个与第一个等式左侧匹配的子项；一旦找到，该子项的所有出现都会被替换为右侧项。如果等式包含变量，则会根据需要进行实例化。要反向使用定理（即作为自右

向左的重写规则），请在定理名称前加上一个短左箭头（←）。如果指定了多个等式，它们将依次被应用。

因此，给定定理 hg : ∀x, g x = f x 和目标 ⊢ h (f a) (g b) (g c)，策略 rw [hg] 会产生子目标 ⊢ h (f a) (f b) (g c)，而 rw [←hg] 则产生子目标 ⊢ h (g a) (g b) (g c)。

除了定理之外，我们还可以指定一个常数的名称。这将尝试使用该常数的定义等式之一作为重写规则。

simp

simp [at *位置*]

simp 策略穷尽地使用一组标准的重写规则（称为 ***simp* 集合**[Simp Set]）来重写目标。simp 集合中的每个等式都用作自左向右的重写规则。simp 集合包含关于预定义符号（例如算术和列表运算符）的各种规则，并且可以通过在适当的定理上添加 @[simp] 属性来扩展。

simp [*定理或常量*$_1$, ..., *定理或常量*$_n$] [at *位置*]

对于上述 simp 变体，指定的定理会被临时添加到 simp 集合中。在定理列表中，星号（*）可以用来代表所有假设。定理名称前的减号（-）会临时将该定理从 simp 集合中移除。一个既能简化假设又能利用结果简化目标靶点的强大命令是 simp [*] at *。

给定定理 hg : ∀x, g x = f x 和目标 ⊢ h (f a) (g b) (g c)，策略 simp [hg] 产生子目标 ⊢ h (f a) (f b) (f c)，其中 g b 和 g c 都已被重写。除了定理之外，我们还可以指定常数的名称。这会临时将常数的定义等式添加到 simp 集合中。

说到这里，你可能会想，「那么 simp 到底做了什么呢?」当然，你可以去研究源代码或查阅科学文献。但这可能不是对你时间最有效的利用。事实上，即使是证明助手的专家用户也并不完全理解他们每天使用的策略的行为。最成功的用户会采取一种轻松、随性的态度，依次尝试各种策略，并研究出现的子目标（如果有的话），看看自己是否走在正确的道路上。

随着你不断地使用 simp 和其他策略，你会对它们擅长处理什么样的目标产生一些直觉。这是互动式定理证明只能通过实践来学习的众多原因之一。通常，你不会确切地理解 Lean 做了什么 —— 为什么一个策略成功了，或者失败了。定理证明有时会非常令人沮丧。《**银河系漫游指南**[The Hitchhiker's Guide to the Galaxy]》封面上的那些醒目而友好的大字所给出的建议在这里同样适用：**不要恐慌**[Don't Panic]。

3.6 数学归纳法证明

Structural induction

`induction` 归纳策略对一个归纳类型的值执行结构归纳。**结构归纳**意味着归纳遵循归纳类型的结构。对于由 `Nat.zero` 和 `Nat.succ` 构造的自然数，结构归纳对应于标准的数学归纳法：要证明 `p n`，只需证明 `p 0` 和 `∀k, p k → p (k + 1)`。借助 `induction`，我们可以推理 **??** 一节中通过递归定义的加法和乘法运算。

加法是通过对其第二个参数进行递归定义的。我们将证明两个定理，`add_zero` 和 `add_succ`，它们为我们提供了在**第一个**参数上递归的替代等式。我们从 `add_zero` 开始：

```
theorem add_zero (n : ℕ) :
    add 0 n = n :=
  by
    induction n with
    | zero        => rfl
    | succ n' ih => simp [add, ih]
```

Induction

归纳策略后面跟着两个由它们对应的构造子标识的子证明。此外，任何特定于子证明的变量或假设都可以明确地在构造子名称之后命名。

第一个情况，标记为 `zero`，对应于基本情况 `⊢ add 0 0 = 0`。第二个情况，标记为 `succ`，对应于归纳步骤

```
n' : ℕ, ih : add 0 n' = n' ⊢ add 0 (Nat.succ n') = Nat.succ n'
```

`succ` 后面指定的名称 `n'` 和 `ih` 分别是我们给 `Nat.succ` 的参数和归纳假设所起的名称。用其他名称也可以，但通常将归纳假设称为 `ih`。

我们可以继续通过结构归纳证明定理：

```
theorem add_succ (m n : ℕ) :
    add (Nat.succ m) n = Nat.succ (add m n) :=
  by
    induction n with
    | zero        => rfl
    | succ n' ih => simp [add, ih]

theorem add_comm (m n : ℕ) :
    add m n = add n m :=
  by
    induction n with
    | zero        => simp [add, add_zero]
```

```
    | succ n' ih => simp [add, add_succ, ih]

theorem add_assoc (l m n : ℕ) :
    add (add l m) n = add l (add m n) :=
  by
    induction n with
    | zero       => rfl
    | succ n' ih => simp [add, ih]
```

一旦我们证明了一个二元运算满足交换律和结合律，就应该让 Lean 的自动化机制，尤其是 ac_rfl 知道这一点。下面的命令即为 add 实现了这一点：

```
instance Associative_add : Std.Associative add :=
  { assoc := add_assoc }

instance Commutative_add : Std.Commutative add :=
  { comm := add_comm }
```

（instance 机制将在 **??** 一章讲解。）下面的例子使用 ac_rfl 策略，在 add 的结合律与交换律意义下进行推理：

```
theorem mul_add (l m n : ℕ) :
    mul l (add m n) = add (mul l m) (mul l n) :=
  by
    induction n with
    | zero       => rfl
    | succ n' ih =>
      simp [add, mul, ih]
      ac_rfl
```

下面给出一些关于如何使用数学归纳法进行证明的提示：

- 通常按照目标中出现的某个函数的定义结构来进行归纳证明是有益的。特别地，如果一个函数是通过对其第 *n* 个参数进行递归来定义的，那么对该参数进行归纳往往是合理的。
- 如果归纳的基本情况难以证明，这往往表明选错了变量，或者需要先证明一些辅助引理。

3.7 归纳策略

我们已经在上面看到了 induction 策略的实际用法。为方便查阅，下面给出它的完整语法。一般而言，本指南正文中介绍的每一种策略，都会在名为「*X* 策略」的小节中附带此类语法描述。

induction

```
induction 项 [with
| 构造子₁ 名字₁ => 策略₁
    ⋮
| 构造子ₙ 名字ₙ => 策略ₙ]
```

induction 策略对指定项执行结构归纳。这会生成与该项类型定义中构造子数量相同的子目标。在对应递归构造子（如 Nat.succ 或 List.cons）的子目标中，归纳假设将作为可用假设出现。可选名字 *名字*$_1$、...和 *名字*$_n$ 用于命名生成的变量或假设。

3.8 清理策略

下面这些策略有助于简化证明目标。目前我们尚未用到它们，但在探索证明思路的过程中，它们会很有帮助。

clear

```
clear 变量或假设₁ ... 变量或假设ₙ
```

只要指定的变量和假设没有在目标的其他任何地方使用，clear 就会移除它们。

rename

```
rename 变量的类型或假设的命题 => 新名字
```

rename 策略会修改变量或假设的名字。

3.9 新引入的 Lean 构造总结

属性

@[simp]	将定理添加到 simp 集

证明命令

by	开始一个策略式的证明

策略

`ac_rfl`	证明 `l = r` 满足交换律和结合律
`apply`	将目标的靶点与定理的结论进行匹配
`assumption`	使用假设证明目标
`clear`	从目标中移除变量或假设
`exact`	使用指定的定理证明目标
`induction`	对归纳类型的变量进行结构归纳
`intro`	将 ∀-量化的变量移入到目标的假设中
`rename`	重命名变量或假设
`rfl`	证明 `l = r` 计算等价
`rw`	使用给定的定理作为重写规则重写一次
`simp`	使用一组预先注册的重写规则穷举重写
`sorry`	表示缺少证明

策略组合子

`·`	专注于第一个子目标；需要证明该子目标。

第四章 4

Forward Proofs

Tactical proofs work backwards. They start from the goal and reduce it to existing theorems and hypotheses. Often it makes sense to work in a forward fashion: to start with existing theorems and hypotheses and proceed step by step towards our goal. *Structured proofs* are a style that supports this kind of reasoning. Tactical proofs tend to be easier to write but harder to read. Most users combine the two styles, using whichever seems the most appropriate for the situation. The higher readability of structured proofs make them popular with some users.

Structured proofs are syntactic sugar sprinkled over Lean's *proof terms*. They are built using keywords such as `assume`, `have`, `let`, and `show` that mimic pen-and-paper proofs. All Lean proofs, whether tactical or structured, are reduced internally to proof terms. We have seen some specimens already, in Chapter **??**: Given hypotheses `ha : a` and `hab : a → b`, the term `hab ha` is a proof term for the proposition `b`, and we write `hab ha : b`. The names of theorems and hypotheses, such as `ha` and `hab`, are also proof terms. Pushing this idea further, given `hbc : b → c`, we can build the proof term `hbc (hab ha)` for the proposition `c`. We can think of `hab` as a function that converts a proof of `a` to a proof of `b`, and similarly for `hbc`.

Structured proofs are the default in Lean. They can be used outside tactic mode. To enter tactic mode, we must use the `by` command.

The concepts covered here are described in more detail in Chapters 2 to 4 of *Theorem Proving in Lean 4* [**?**]. Nederpelt and Geuvers's textbook [**?**] is another useful reference.

4.1 Structured Proofs

As a first example, consider the following structured proof:

```
theorem fst_of_two_props :
    ∀a b : Prop, a → b → a :=
  fix a b : Prop
  assume ha : a
  assume hb : b
  show a from
    ha
```

Each variable bound by a ∀ quantifier and each assumption of an implication is introduced explicitly in the proof using the `fix` and `assume` commands. Several variables can be introduced simultaneously. We will often omit the types of variables, especially when they can be guessed from their names; however, we will always spell out the propositions and put them one per line, to increase readability—which is, after all, one of the main potential advantages of the structured style. The `show ... from` command at the end repeats the proposition to prove, for the sake of readability, and gives the proof after the keyword `from`. The goal at this point is `a b : Prop, ha : a, hb : b ⊢ a`.

Informally, we could write the proof as follows:

> Fix some propositions a and b.
>
> Assume (ha) a and (hb) b are true.
>
> We must show a. This follows trivially from ha. □

Some authors would insert qualifiers such as "arbitrary but fixed" in front of "propositions," or they would write "Let a and b be some propositions." All these variants are equivalent. And instead of "□," we could write "QED" to conclude the proof.

The Lean proof above is atypical in that the goal's target appears among the hypotheses. Usually, we must perform some intermediate reasoning steps, essentially of the form "from so-and-so, we have such-and-such." In Lean, each intermediate step takes the form of a `have` command, as in the following example:

```
theorem prop_comp (a b c : Prop) (hab : a → b) (hbc : b
    → c) :
    a → c :=
  assume ha : a
  have hb : b :=
```

```
    hab ha
  have hc : c :=
    hbc hb
  show c from
    hc
```

Informally:

> Assume (ha) a is true.
>
> From ha and hab, we have (hb) b.
>
> From hb and hbc, we have (hc) c.
>
> We must show c. This follows trivially from hc. □

Notice that this is a forward proof: It progresses one theorem at a time from the hypothesis a to the desired theorem c. Modus ponens ("from a and a → b, derive b") is expressed by simple juxtaposition (e.g., hab ha).

In general, the fix–assume–show skeleton repeats the theorem's statement. We often name the fixed variables after the bound variables from the goal, as we did here, but this is not mandatory. Moreover, it is good practice to avoid shadowing existing variables. Between the last fix or assume and the show, we can have as many have commands as we want, depending on how detailed we want the argument to be. Details can increase readability, but providing too many can overwhelm the reader.

The have command has a similar syntax to theorem but appears inside a structured proof. We can also think of a have as a definition. In have hb : b := hab ha, the right-hand side hab ha is a proof term for b, and the left-hand side hb is defined as a synonym for that proof term. From then on, hb and hab ha can be used interchangeably. Since hb and hc are used only once and their proofs are very short, experts would tend to inline them, replacing hc by hbc hb and then hb by hab ha, yielding

```
theorem prop_comp_inline (a b c : Prop) (hab : a → b)
    (hbc : b → c) :
  a → c :=
  assume ha : a
  show c from
    hbc (hab ha)
```

A typical structured proof has the following fix–assume–have–show format:

```
theorem hr :
    ∀(c₁ : σ₁) ... (c? : σ?), P₁ → ⋯ → P? → R :=
  fix (c₁ : σ₁) ... (c? : σ?)
  assume h₁ : P₁
    ?
  assume h? : P?
  have k₁ : Q₁ := ...
    ?
  have k? : Q? := ...
  show R from ...
```

4.2 Structured Constructs

The previous section presented the main commands for writing structured proofs: `fix`, `assume`, `have`, and `show`. We now review the components of structured proofs more systematically.

Theorem or Hypothesis

The simplest structured proof, apart from `sorry`, is the name of a theorem or hypothesis. If we have

```
theorem two_add_two_Eq_four :
    2 + 2 = 4 :=
  by ...
```

then the theorem name `two_add_two_Eq_four` can be used as a proof of `2 + 2 = 4` later. For example:

```
theorem this_time_with_feeling :
    2 + 2 = 4 :=
  two_add_two_Eq_four
```

We can pass arguments to theorems to instantiate ∀ quantifiers and to discharge assumptions. Suppose the theorem `add_comm (m n : ℕ) : add m n = add n m` is available, and suppose we want to prove its instance `add 0 n = add n 0`. This can be achieved neatly using the name of the theorem and two arguments:

```
theorem add_comm_zero_left (n : ℕ) :
    add 0 n = add n 0 :=
  add_comm 0 n
```

This is equivalent to the tactical proof `by exact add_comm 0 n` but is more concise. The `exact` tactic can be seen as the inverse of `by`. Why enter tactic mode only to leave it immediately?

As with `exact` and `apply`, the theorem or hypothesis's statement is matched with the current goal up to computation. This gives some flexibility.

fix

```
fix names : type
```

The `fix` command moves ∀-quantified variables from the goal's target to the local context. It is a structured version of the `intro` tactic.

Note that the standard Lean tactic for fixing variables is `fun`. We prefer to use `fix`, which is provided by `LoVelib`, as a more readable alternative. We will keep `fun` for specifying anonymous functions.

assume

```
assume name : proposition
```

The `assume` command moves the leading assumption from goal's target to the local context. It can be seen as a structured version of the `intro` tactic.

Note that the standard Lean tactic for stating assumptions is `fun`. We prefer to use `assume`, which is provided by `LoVelib`, as a more readable alternative. We will keep `fun` for specifying anonymous functions.

have

```
have name : proposition :=
  proof
```

The `have` command lets us state and prove an intermediate theorem, which may refer to names introduced by previous `fix`es, `assume`s, and `have`s. The proof can be tactical or structured. Generally, we tend to use structured proofs to sketch the main argument and resort to tactical proofs for proving subgoals or uninteresting intermediate steps. Another kind of mixture arises when we pass arguments to theorem names. For example, given `hab : a → b` and `ha : a`, the tactic `exact hab ha` will prove the goal ⊢ b. Here, `hab ha` is a proof term nested inside a tactic.

let

```
let name [: type] := term
```

The `let` command introduces a new local definition. It can be used to name a complex object that occurs several times in the proof afterwards. It is similar to `have` but is designed for computable data, not proofs. Expanding or introducing a `let` corresponds to ζ-conversion (Section **??**).

The construct `let name [: type] := term` must be followed by a line break or a semicolon (`;`).

show

```
show proposition from
  proof
```

The `show` command lets us repeat the goal to prove, which can be useful as documentation. It also allows us to rephrase the goal in a syntactically equal form up to computation. Instead of the syntax `show proposition from proof`, we can simply write `proof` if we do not want to repeat the goal and do not need to rephrase it. The proof can be tactical or structured.

4.3 Forward Reasoning about Connectives and Quantifiers

Reasoning about the logical connectives and quantifiers in a forward fashion uses the same introduction and elimination rules as in tactic mode (Section **??**). A few examples will give the flavor. Let us start with conjunction:

```
theorem And_swap (a b : Prop) :
    a ∧ b → b ∧ a :=
  assume hab : a ∧ b
  have ha : a :=
    And.left hab
  have hb : b :=
    And.right hab
  show b ∧ a from
    And.intro hb ha
```

Even readers who do not know what `And.left` etc. mean can understand that we extract a and b from a ∧ b and put them back together as b ∧ a. Mathematicians

would probably have an easier time making sense of this proof than of its tactical counterpart:

```
theorem And_swap_tactical (a b : Prop) :
    a ∧ b → b ∧ a :=
  by
    intro hab
    apply And.intro
    apply And.right
    exact hab
    apply And.left
    exact hab
```

In general, backward proofs are easier to derive "mindlessly," and most of the automation provided by proof assistants works backwards. This makes sense: Pretend that you are Miss Marple or Hercule Poirot on a murder investigation. A backward investigation would start from the crime scene and try to extract clues that potentially connect a handful of suspects to the crime. In contrast, a forward investigation might start by questioning as many as eight billion people to determine whether they have an alibi. Which approach is more likely to succeed?

Our next examples concern *one-point rules*. These are theorems that can be used to eliminate a quantifier when the bound variable can effectively take only one value. For example, the proposition `∀n, n = 666 → beast ≥ n` can be simplified to `beast ≥ 666`. The following theorem justifies this simplification:

```
theorem Forall.one_point {α : Type} (t : α) (P : α →
   Prop) :
    (∀x, x = t → P x) ↔ P t :=
  Iff.intro
    (assume hall : ∀x, x = t → P x
     show P t from
       by
         apply hall t
         rfl)
    (assume hp : P t
     fix x : α
     assume heq : x = t
     show P x from
       by
         rw [heq]
```

```
            exact hp)
```

The proof may look intimidating, but it was not hard to develop. The key was to proceed one step at a time. At first, we observed that the goal is an equivalence, so we wrote

```
Iff.intro (_) (_)
```

where `Iff.intro` is the introduction rule for ↔ (Section **??**). The two placeholders are important to make this proof well formed. We use parentheses because we strongly suspect that the subproofs will be nontrivial. Because proofs are basically terms, which are basically programs, the advice we gave in Section **??** applies here as well, mutatis mutandis:

> 关键在于，proof 应始终保持语法正确。我们在 Visual Studio Code 中唯一应该看到的红色下划线应该出现在占位符下方。一般来说，开发 proof 的一个好的原则是，从可以编译的 proof 开始，进行尽可能小的更改，以获得新的可以编译的 proof，并重复此过程，直到 proof 完成。

Hovering over the first placeholder makes the corresponding subgoal appear. We can see that Lean expects a proof of `⊢ (∀x, x = t → P x) → P t`, so we provide a suitable skeleton. A structured proof of an implication consists of an `assume` followed by a `show`:

```
Iff.intro
  (assume hall : ∀x, x = t → P x
   show P t from
     _)
  (_)
```

Each of the remaining placeholders can be replaced by a structured proof or a tactical proof. To eliminate the first placeholder, we enter tactic mode and apply the instance `t = t → P t` of the assumption `∀x, x = t → P x` to the goal `⊢ P t`. This leaves the proof obligation `t = t`, which we dischage using the `rfl` tactic:

```
Iff.intro
  (assume hall : ∀x, x = t → P x
   show P t from
     by
       apply hall t
       rfl))
  (_)
```

The second placeholder stands for ⊢ P t → ∀x, x = t → P x. To eliminate it, we write assume corresponding to the first implication's assumption, fix for ∀, assume for the second implication's assumption, and show for the conclusion:

```
(assume hp : P t
 fix x : α
 assume heq : x = t
 show P x from
   _)
```

The show's proof is tactical: We rewrite x to t in ⊢ P x, obtaining ⊢ P t, which matches the assumption hp:

```
(assume hp : P t
 fix x : α
 assume heq : x = t
 show P x from
   by
     rw [heq]
     exact hp)
```

Let us check that the one-point rule actually works on our motivating example:

```
theorem beast_666 (beast : ℕ) :
    (∀n, n = 666 → beast ≥ n) ↔ beast ≥ 666 :=
  Forall.one_point _ _
```

It works. Instead of Forall.one_point _ _, we could also have written Forall.one_point 666 (fun m ↦ beast ≥ m).

Finally, the one-point rule for ∃ demonstrates how to use the introduction and elimination rules for ∃ in a structured proof:

```
theorem Exists.one_point {α : Type} (t : α) (P : α →
   Prop) :
    (∃x : α, x = t ∧ P x) ↔ P t :=
  Iff.intro
    (assume hex : ∃x, x = t ∧ P x
     show P t from
       Exists.elim hex
         (fix x : α
          assume hand : x = t ∧ P x
          have hxt : x = t :=
            And.left hand
```

```
            have hpx : P x :=
              And.right hand
            show P t from
              by
                rw [←hxt]
                exact hpx))
      (assume hp : P t
       show ∃x : α, x = t ∧ P x from
         Exists.intro t
           (have tt : t = t :=
              by rfl
            show t = t ∧ P t from
              And.intro tt hp))
```

Although the proof might look overwhelming, it was developed mostly without thinking using the same procedure as for the proof of the one-point rule for ∀.

First, we use the introduction rule for ↔:

```
Iff.intro (_) (_)
```

For the first placeholder, we must provide a proof of `P t` under the assumption `∃x : α, x = t ∧ P x`. Thus, we expand the proof skeleton to

```
Iff.intro
  (assume hex : ∃x : α, x = t ∧ P x
   show P t from
     _)
  (_)
```

At this point, we want to extract the witness for `x` in the assumption `hex`. To achieve this, we use the elimination rule for ∃:

```
Iff.intro
  (assume hex : ∃x : α, x = t ∧ P x
   show P t from
     Exists.elim hex (_))
  (_)
```

The first placeholder asks for a proof of the subgoal `⊢ ∀x, x = t ∧ P x → P t`. We sketch one:

```
Iff.intro
  (assume hex : ∃x : α, x = t ∧ P x
```

```
    show P t from
      Exists.elim hex
        (fix x : α
         assume hand : x = t ∧ P x
         show P t from
           _))
  (_)
```

Notice how the subproof mirrors the subgoal, with `fix` corresponding to ∀, `assume` corresponding to →'s assumption, and `show` corresponding to →'s conclusion. We can now access the witness `x` and its characteristic property `x = t ∧ P x`.

Next, we extract the two conjuncts of `x = t ∧ P x`, using the `have` command and the elimination rules for ∧ (`And.left` and `And.right`):

```
Iff.intro
  (assume hex : ∃x : α, x = t ∧ P x
   show P t from
     Exists.elim hex
       (fix x : α
        assume hand : x = t ∧ P x
        have hxt : x = t :=
          And.left hand
        have hpx : P x :=
          And.right hand
        show P t from
          _))
  (_)
```

Now we can attempt a proof of `P t`. We rewrite `t` into `x` and apply the hypothesis `P x`:

```
Iff.intro
  (assume hex : ∃x : α, x = t ∧ P x
   show P t from
     Exists.elim hex
       (fix x : α
        assume hand : x = t ∧ P x
        have hxt : x = t :=
          And.left hand
        have hpx : P x :=
```

```
            And.right hand
          show P t from
            by
              rw [←hxt]
              exact hpx))
(_)
```

For the remaining placeholder, we first provide a proof skeleton based on the subgoal ⊢ P t → ∃x : α, x = t ∧ P x:

```
(assume hp : P t
 show ∃x : α, x = t ∧ P x from
   _)
```

Then we use the introduction rule for ∃ to provide the witness t:

```
(assume hp : P t
 show ∃x : α, x = t ∧ P x from
   Exists.intro t (_))
```

Then comes more boilerplate:

```
(assume hp : P t
 show ∃x : α, x = t ∧ P x from
   Exists.intro t
     (show t = t ∧ P t from
        _))
```

The last conjunction, t = t ∧ P t, is easy to prove: Its left-hand side follows by reflexivity, and its right-hand side is simply the assumption hp. Finally, we get

```
(assume hp : P t
 show ∃x : α, x = t ∧ P x from
   Exists.intro t
     (have tt : t = t :=
        by rfl
      show t = t ∧ P t from
        And.intro tt hp))
```

To develop the above structured proofs, we used largely the same procedure as for exhibiting an inhabitant of a type (Section **??**), understanding the function arrow as implication. The similarity between the two procedures is no coincidence, as we will see in Section **??**.

4.4 Calculational Proofs

In informal mathematics, we often express proofs as transitive chains of equalities, inequalities, or equivalences (e.g., a = b = c, a ≥ b ≥ c, or a ↔ b ↔ c). In Lean, such *calculational proofs* are supported by the `calc` command, which provides a lightweight syntax and takes care of applying transitivity theorems for preregistered relations, such as equality and the arithmetic comparison operators.

The general syntax is as follows:

```
calc
  term₀ op₁ term₁ :=
    proof₁
  _ op₂ term₂ :=
    proof₂
      ⋮
  _ opₙ termₙ :=
    proofₙ
```

The underscores (_) are part of the syntax. Each $\texttt{proof}_i$ justifies the statement $\texttt{term}_{i-1}\ \texttt{op}_i\ \texttt{term}_i$. The operators $\texttt{op}_i$ need not be identical, but they must be compatible with each other. For example, $=$, $<$, and $\leq$ are compatible, whereas $>$ and $<$ are not.

A simple example follows:

```
theorem two_mul_example (m n : ℕ) :
    2 * m + n = m + n + m :=
  calc
    2 * m + n = m + m + n :=
      by rw [Nat.two_mul]
    _ = m + n + m :=
      by ac_rfl
```

Mathematicians (assuming they would condescend to offer a justification for such a trivial result) could have written the above proof roughly as follows:

$$\begin{aligned} 2 * m + n &= (m + m) + n && \text{(since } 2 * m = m + m\text{)} \\ &= m + n + m && \text{(by associativity and commutativity of +)} \end{aligned}$$

□

In the Lean proof, the underscore stands for the term `(m + m) + n`, which we would have had to repeat had we written the proof without `calc`:

```
theorem two_mul_example_have (m n : ℕ) :
    2 * m + n = m + n + m :=
  have hmul : 2 * m + n = m + m + n :=
    by rw [Nat.two_mul]
  have hcomm : m + m + n = m + n + m :=
    by ac_rfl
  show _ from
    Eq.trans hmul hcomm
```

Notice that with `have`s, we also need to explicitly invoke `Eq.trans` and to give names to the two intermediate steps.

4.5 Forward Reasoning with Tactics

Many users prefer the tactic mode to structured proofs. But even in tactic mode, it can be useful to reason in a forward fashion, mixing forward and backward reasoning steps. The structured proof commands `have`, `let`, and `calc` are also available as tactics, making this possible.

The following example demonstrates the `have` and `let` tactics on a theorem we have seen several times already:

```
theorem prop_comp_tactical (a b c : Prop) (hab : a → b)
    (hbc : b → c) :
    a → c :=
  by
    intro ha
    have hb : b :=
      hab ha
    let c' := c
    have hc : c' :=
      hbc hb
    exact hc
```

have

```
have name : proposition :=
  proof
```

The `have` tactic lets us state and prove an intermediate theorem in tactic mode. Afterwards, the theorem is available as a hypothesis in the goal state.

let

```
let name [: type] := term
```

The `let` tactic lets us introduce a local definition in tactic mode. Afterwards, the defined symbol and its definition are available in the goal state.

calc

```
calc
  term₀ op₁ term₁ :=
    proof₁
  _ op₂ term₂ :=
    proof₂
      ⋮
  _ opₙ termₙ :=
    proofₙ
```

The `calc` tactic lets us enter a calculational proof (Section **??**) in tactic mode. The tactic has the same syntax as the structured proof command of the same name. We can regard `calc` ..., where `calc` is a tactic, as an alias for `apply (exact calc` ...), where `calc` is the structured proof command described above.

4.6 Dependent Types

Dependent types are the defining feature of the *dependent type theory* family of logics. Although you may not be familiar with the terminology, you are likely to be familiar with the concept in some form or other.

Consider a function `pick` that takes a natural number `n` (i.e., a value from `ℕ = {0, 1, 2, ...}`) and that returns a natural number between `0` and `n`. Intuitively, `pick n` should have the type `{0, 1, ..., n}` (i.e., the type consisting of all natural numbers `i ≤ n`). In Lean, this type is written `{i : ℕ // i ≤ n}`. This would be the type of `pick n`. Mathematically inclined readers might want to think of `pick` as an `ℕ`-indexed family of terms

$$(\texttt{pick n : \{i : ℕ // i ≤ n\}})_{\texttt{n:ℕ}}$$

in which the type of each term depends on the index—e.g., `pick 5 : {i : ℕ // i ≤ 5}`. But what would be the type of `pick` itself? We would like to express that

`pick` is a function that takes an argument $n : \mathbb{N}$ and that returns a value of type $\{i : \mathbb{N} \mathbin{//} i \leq n\}$. To capture this, we will write

$$\texttt{pick} : (n : \mathbb{N}) \to \{i : \mathbb{N} \mathbin{//} i \leq n\}$$

This is a dependent type: The type of the result depends on the value of the argument n. (The variable name n itself is immaterial; we could also write m or x.)

Unless otherwise specified, a *dependent type* means a type depending on a (non-type) term, as above, with $n : \mathbb{N}$ as the term and $\{i : \mathbb{N} \mathbin{//} i \leq n\}$ as the type that depends on it. But:

- A type may also depend on another type—for example, the type constructor `List`, its η-expanded variant $\texttt{fun}\ \alpha : \texttt{Type} \mapsto \texttt{List}\ \alpha$, or the polymorphic type $\texttt{fun}\ \alpha : \texttt{Type} \mapsto \alpha \to \alpha$ of functions with the same domain and codomain.
- A term may depend on a type—for example, the polymorphic identity function $\texttt{fun}\ \alpha : \texttt{Type} \mapsto \texttt{fun}\ x : \alpha \mapsto x$.
- And of course, a term may also depend on a term—for example, $\texttt{fun}\ n : \mathbb{N} \mapsto n + 2$.

In summary, there are four cases for $\texttt{fun}\ x \mapsto t$:

Body (t)		Argument (x)	Description
A term	depending on	a term	Simply typed λ-expression
A type	depending on	a term	Dependent type (in the narrow sense)
A term	depending on	a type	Polymorphic term
A type	depending on	a type	Polymorphic type constructor

The last three rows correspond to the three axes of Henk Barendregt's λ-cube.[1]

The App and Fun rules presented in Section **??** must be generalized to work with dependent types:

$$\frac{C \vdash t : (x : \sigma) \to \tau[x] \qquad C \vdash u : \sigma}{C \vdash t\ u : \tau[u]}\ \textsc{App}'$$

$$\frac{C, x : \sigma \vdash t : \tau[x]}{C \vdash (\texttt{fun}\ x : \sigma \mapsto t) : (x : \sigma) \to \tau[x]}\ \textsc{Fun}'$$

The notation $\tau[x]$ stands for a type that may contain x, and $\tau[u]$ stands for the same type where all occurrences of x have been replaced by u.

[1] https://en.wikipedia.org/wiki/Lambda_cube

The simply typed case arises when x does not occur in $\tau[x]$. Then, we can simply write $\sigma \to$ instead of $(x : \sigma) \to$. The familiar notation $\sigma \to \tau$ is equivalent to $(_ : \sigma) \to \tau$. It is easy to check that App′ and Fun′ coincide with App and Fun when x does not occur in $\tau[x]$.

The example below demonstrates App′:

$$\frac{\vdash \texttt{pick} : (n : \mathbb{N}) \to \{i : \mathbb{N} \mathbin{//} i \leq n\} \qquad \vdash 5 : \mathbb{N}}{\vdash \texttt{pick}\ 5 : \{i : \mathbb{N} \mathbin{//} i \leq 5\}}\ \textsc{App}'$$

The next example demonstrates Fun′:

$$\frac{\dfrac{\alpha : \texttt{Type},\ x : \alpha \vdash x : \alpha}{\alpha : \texttt{Type} \vdash (\texttt{fun}\ x : \alpha \mapsto x) : \alpha \to \alpha}\ \textsc{Fun or Fun}'}{\vdash (\texttt{fun}\ \alpha : \texttt{Type} \mapsto \texttt{fun}\ x : \alpha \mapsto x) : (\alpha : \texttt{Type}) \to \alpha \to \alpha}\ \textsc{Fun}'$$

The picture is incomplete because we only check that the terms—the entities on the left-hand side of a colon (`:`)—are well typed. The types—the entities on the right-hand side of a colon—should also be checked, using the same type system. For example, the type of `Nat.succ` is $\mathbb{N} \to \mathbb{N}$, whose type is `Type`. Types of types, such as `Type` and `Prop`, are called universes. We will study them more closely in Chapter **??**.

Remarkably, universal quantification is simply an alias for a dependent type: $\forall x : \sigma,\ \tau$ abbreviates $(x : \sigma) \to \tau$. This will become clearer below.

4.7 The PAT Principle

You will likely have noticed that the same symbol $\to$ is used both for implication (e.g., `False` $\to$ `True`) and as the type constructor of functions (e.g., $\mathbb{Z} \to \mathbb{N}$). Without context, we cannot tell whether $a \to b$ refers to the type of a function with domain a and codomain b or to the proposition "a implies b."

It turns out that the two concepts not only *look* the same, they *are* the same. This is called the *PAT principle*, where PAT is a double mnemonic:

PAT = propositions as types PAT = proofs as terms

Furthermore, because types are also terms, it also means that propositions are terms. However, PAT is not a quadruple mnemonic (~~PAT = proofs as types~~). Also note that not all types are propositions, and not all terms are proofs.

By using terms and types to represent proofs and propositions, dependent type theory achieves a considerable economy of concepts. The question

"Is H a proof of P?"

becomes equivalent to

"Does the term H have type P?"

As a result, inside of Lean, there is no proof checker, only a type checker. Proofs are checked by the type checker.

Let us review the dramatis personae one by one. We use the mathematical variables σ, τ for types; P, Q for propositions; t, u, x for terms; and h, G, H, for proofs.

Starting with "propositions as types," for types, we have the following:

- σ → τ is the type of functions from σ to τ.
- (x : σ) → τ[x] is the dependent type of functions from x : σ to τ[x].

In contrast, for propositions, we have the following:

- P → Q can be read as "P implies Q," or as the type of functions mapping proofs of P to proofs of Q.
- ∀x : σ, Q[x] can be read as "for all x, Q[x]," or as the type of functions of type (x : σ) → Q[x], mapping values x of type σ to proofs of Q[x].

Continuing with "proofs as terms," for terms, we have the following:

- A constant is a term.
- A variable is a term.
- t u is the application of function t to argument u.
- fun x ↦ t[x] is a function mapping x to t[x].

In contrast, for proofs (i.e., proof terms), we have the following:

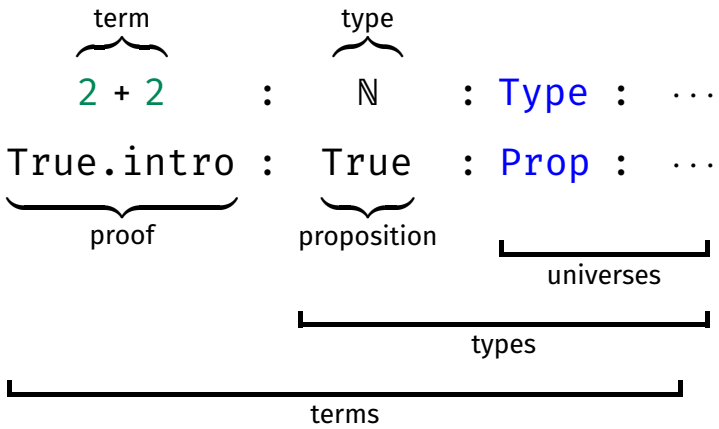
- The name of a theorem or hypothesis is a proof.
- H t, which instantiates the leading ∀ quantifier of proof H's statement with term t, is a proof.
- H G, which discharges the leading assumption of H's statement with proof G, is a proof. This operation is called modus ponens.
- fun h : P ↦ H[h] is a proof of P → Q, assuming H[h] is a proof of Q for all h : P.
- fun x : σ ↦ H[x] is a proof of ∀x : σ, Q[x], assuming H[x] is a proof of Q[x] for all x : σ.

The last two cases are justified by the Fun′ rule. In a structured proof, as opposed to a raw proof term, we would write assume or fix instead of fun, and we would probably want to repeat the conclusion using show for readability, as follows:

```
theorem case_4 :
    P → Q :=
  assume h : P
  show Q from
    H[h]
```

```
theorem case_5 :
    ∀x : σ, Q[x] :=
  fix x : σ
  show Q[x] from
    H[x]
```

The terminology of dependent type theory can be quite confusing, because some words have a narrow and a broad sense. The following diagram captures the various meanings of important words:



In the broad sense, any expression is a term, any expression that may occur on the right-hand side of a typing judgment is a type, and any expression that may occur on the right-hand side of a typing judgment with a type on its left-hand side is a universe. This is consistent with the reading of `t : u` as "`t` has type `u`" and the notion that universes are types of types.

Some Lean commands have two names but are essentially the same by the PAT principle. This is the case for `fix` and `assume`; indeed, both are `LoVelib` aliases for `fun`. There are also pairs with slightly different behavior, such as `def`/`theorem` and `let`/`have`. The fundamental difference is this: When we define some function or data, we care not only about the type but also about the body—the behavior. On the other hand, once we have proved a theorem, the proof itself becomes *irrelevant*. All that matters is that there *is* a proof. We will return to the topic of proof irrelevance in Chapter **??**.

The following table summarizes the differences between tactical proofs, structured proofs, and raw proof terms:

Tactical proof	Structured proof	Raw proof term
`intro x`	`fix x : τ`	`fun x ↦`
`intro h`	`assume h : P`	`fun h ↦`
`have k := H`	`have k := H`	`(fun k ↦ ...) H`
`let x := t`	`let x := t`	`(fun x ↦ ...) t`
`exact (H : P)`	`show P from H`	`H : P`
`calc`	`calc`	`calc`

Notice that `let x := t` followed by u is essentially a way of writing `(fun x ↦ u) t`, except that β-reduction is disabled.

4.8 Induction by Pattern Matching and Recursion

In Section **??**, we reviewed the use of the `induction` tactic to perform proofs by induction. An alternative, more flexible style relies on pattern matching and the PAT principle.

Recall the definition of the reverse of a list from the end of Section **??**:

```
def reverse {α : Type} : List α → List α
  | []       => []
  | x :: xs => reverse xs ++ [x]
```

In fact, `reverse` exists as `List.reverse` in Lean's standard library, with a more efficient but less convenient definition for reasoning. A useful property to prove is that `reverse` is its own inverse: `reverse (reverse xs) = xs` for all lists `xs`. However, if we try to prove it by induction, we quickly run into an obstacle. The induction step is

```
: ∀xs, reverse (reverse xs) = xs ⊢ reverse (reverse xs ++ [x]) = x ::
xs ih
```

Notice the unpleasant presence of `++ [x]` inside the double `reverse` sandwich. We need a way to "distribute" the outer `reverse` over `++` to obtain a term that matches the induction hypothesis's left-hand side. The trick is to prove and use the following theorem:

```
theorem reverse_append {α : Type} :
    ∀xs ys : List α,
      reverse (xs ++ ys) = reverse ys ++ reverse xs
  | [],      ys => by simp [reverse]
  | x :: xs, ys => by simp [reverse, reverse_append xs]
```

The theorem's proof arguably looks more like a recursive function definition than a proof. The patterns on the left, `[]` and `x :: xs`, correspond to the two constructors of the ∀-quantified variable `xs`. On the right-hand side of each `=>` symbol is a proof for the corresponding case. The variables on which we can pattern-match are those that appear in the ∀ quantifiers, in order of occurrence (here, `xs` and `ys`). Inside the induction step's proof, the induction hypothesis is available under the same name as the theorem we are proving (`reverse_append`).

We explicitly pass `xs` as an argument to the induction hypothesis. This restricts the hypothesis so that it only applies to `xs` but not to other lists. In particular, this ensures that the theorem will not be applied to `x :: xs`, which could lead to a circular proof: "To prove `reverse_append (x :: xs)`, use `reverse_append (x :: xs)`." Lean's termination checker would notice that the proof is ill-founded and raise an error, but we want to avoid that. In addition, the explicit argument `xs` is useful as documentation, effectively saying, "The only recursive instance of the theorem we need is the one on `xs`."

For reference, a tactical proof would be as follows:

```
theorem reverse_append_tactical {α : Type} (xs ys : List
    α) :
    reverse (xs ++ ys) = reverse ys ++ reverse xs :=
  by
    induction xs with
    | nil           => simp [reverse]
    | cons x xs' ih => simp [reverse, ih]
```

The theorem would also be provable, and useful, if we put `[y]` instead of `ys`. But it is a good habit to state theorems as generally as possible. This results in more reusable libraries. Moreover, this is often necessary to obtain a strong enough induction hypothesis in a proof by induction. In general, finding the right inductions and theorems can require thought and creativity.

Simultaneous pattern matching on multiple variables is supported (e.g., `xs` and `ys` above). The patterns are then separated by commas. The general format is

```
theorem name (params₁ : type₁) ... (paramsₘ : typeₘ) :
    statement
  | patterns₁ => proof₁
      ⋮
  | patternsₙ => proofₙ
```

Notice the strong similarity with the syntax of `def` (Section **??**). The two commands

are, in fact, almost the same, but `theorem` considers the defined term or the proof opaque, whereas `def` keeps it transparent. Since the actual proofs are irrelevant once a theorem is proved (Section **??**), there is no need to expand them later. A similar distinction exists between `let` and `have`.

By the PAT principle, a proof by induction by pattern matching and recursion is the same as a recursive proof term. When we invoke the induction hypothesis, we are really just invoking a recursive function recursively. This explains why the induction hypothesis has the same name as the theorem we prove. Lean's termination checker is used to establish well-foundedness of the proof by induction.

With the `reverse_append` theorem in place, we can return to our initial goal:

```
theorem reverse_reverse {α : Type} :
    ∀xs : List α, reverse (reverse xs) = xs
  | []      => by rfl
  | x :: xs =>
    by simp [reverse, reverse_append, reverse_reverse xs
   ]
```

Induction by pattern matching and recursion is popular among Lean users. Its main advantages are its convenient syntax and its support for well-founded induction, which is more powerful than structural induction as provided by the `induction` tactic (Section **??**). However, in this guide, we will not need the full power of well-founded induction. Furthermore, for subtle logical reasons, induction by pattern matching and recursion is not available for inductive predicates, which are the topic of Chapter **??**. For these reasons, we will often prefer the `induction` tactic.

4.9 Summary of Newly Introduced Lean Constructs

Proof Commands

`assume`	states assumptions
`calc`	combines proofs by transitivity
`fix`	fixes variables
`have`	states an intermediate theorem
`let`	introduces a local definition
`show`	states the target

Tactics

`calc`	combines proofs by transitivity
`have`	states an intermediate theorem
`let`	introduces a local definition

第二部分

函数式与逻辑编程

第三部分

程序语义

第四部分

数学